zkAgent: Verifiable Agent Execution via One-Shot Complete LLM Inference Proof

Lizheng Wang
Shanghai Jiao Tong University

Hancheng Lou
Shanghai Jiao Tong University

Chongrong Li
Shanghai Jiao Tong University

Yu Yu
Shanghai Jiao Tong University

Yuncong Hu
Shanghai Jiao Tong University

Abstract

Recent advances in large language models have enabled LLM-based agents to move beyond text generation toward long-term execution involving tool use, multi-step interactions, and autonomous decision-making. However, the agent provider may be compromised and return malicious outputs. As agents increasingly manage sensitive data and financial assets, such misbehavior can cause severe real-world harm. Recent work leverages zero-knowledge proofs to verify the correctness of LLM inference, ensuring that the provider can only return outputs consistent with the claimed model. Nevertheless, these approaches are limited to standalone transformer computations and fail to support agent executions.

We present zkAgent, an efficient SNARK system for agent execution. Beyond prior Transformer-only proofs, zkAgent proves the entire agent execution, including end-to-end LLM inference and tool interactions. Furthermore, zkAgent achieves scalable proof generation by proving multi-step agent interactions through a single, one-shot inference proof, eliminating the need to prove each intermediate token generation. To our knowledge, zkAgent is the first system to provide practical verifiable *agent execution* that simultaneously attests to complete LLM inference and tool interactions.

Our evaluation shows that, for a 512-token agent inference with GPT-2, zkAgent achieves an amortized proving time of 1.05 s/token, a 294× speedup over the state of the art, zkGPT (USENIX Security '25) which requires 309 s/token with step-by-step generation. zkAgent also reduces verification time by 9,690× (0.45 s vs. 4361.09 s). Moreover, for end-to-end agent executions, such as a weather agent and a coding assistant, zkAgent completes proving in 240 s and verification in about 0.5 s, with proof size of 42MB, making verifiable agent execution practical in real-world deployments.

1 Introduction

Large language model (LLM)–based agents [44, 78, 82] are increasingly deployed as interactive assistants in web and application interfaces, helping users reason over complex information, and invoke external tools. As such agents are entrusted greater responsibility—ranging from decision support to automated execution —the correctness of both reasoning processes and the resulting actions becomes critical. For example, emerging open-source deployments—such as self-hosted assistants like openclaw [65]—together with standardization efforts and recent developments [22] in the Ethereum ecosystem, are increasingly enabling agents to invoke business workflows, operate user-controlled resources, and even initiate payments in supported settings [52].

This raises an *end-to-end integrity problem* for *agent executions*. When inference and execution are outsourced to third-party, a malicious or economically incentivized service provider may violate agent integrity in multiple ways [1, 13, 58, 61]: (i) by silently substituting the advertised model with a cheaper or backdoored alternative; (ii) by deviating from the prescribed agent procedure, such as truncating the model generation prematurely to reduce cost; or (iii) by tampering with *tool observations*, such as forging web-search results, censoring or injecting facts, or misreporting sandbox execution outputs. As agents extend LLMs from text generation to automated execution even in high-stakes applications, the impact of such integrity violations is no longer limited to incorrect answers but can propagate through multi-step workflows, amplifying their real-world consequences.

Recent work on *zero-knowledge proofs for large language models* [14, 33, 46, 58, 60, 70] (zkLLM) represents an initial step toward mitigating integrity concerns in outsourced LLM inference. These approaches leverage SNARK techniques [5, 7, 17, 29, 30] to ensure that a claimed model was executed correctly on a given input. At a high level, zkLLM systems encode the forward pass of an LLM as a verifiable computation and generate succinct proofs for the output. These works show the feasibility of verifiable LLM execution.

Despite recent progress, existing zkLLM approaches [14, 58, 60, 70] remain insufficient for verifiable agent execution. They primarily focus on the correctness of the model's internal forward computation (e.g., Transformer-block evaluation).

In contrast, agent executions include over multiple rounds of inference, with prompts and state updated iteratively and interleaved with external tool calls and their observations. This gap highlights several fundamental challenges in achieving end-to-end integrity for agent executions.

Challenge 1: Proving complete inference beyond Transformer blocks. Existing zkLLM systems [14, 33, 46, 58, 60, 70] mostly focus on reducing proof generation cost for Transformer-block computations. Accordingly, they always begin the proved computation from *tensor inputs* (e.g., token embeddings) instead of a discrete user prompt. In other words, token-to-embedding lookup [9, 76], positional encoding [66, 76] (e.g., RoPE), and the final logit-to-token decoding step [34, 71] are typically left outside the scope of these works.

Designing ZK protocols that cover these *non-Transformer* components—embedding lookup, positional encoding, and decoding—is technically challenging for several reasons. First, in the embedding stage, each discrete token must be mapped to a high-dimensional vector via a vocabulary-sized embedding table, which is extremely large in modern LLMs. Generic primitives used in prior zkLLMs—including lookup arguments [28, 32] and offline memory-checkings [63, 64]—typically incur proving overhead that scales linearly in the size of the committed table, making them impractical. Second, positional encoding is not always friendly to arithmetization. For example, rotary positional encoding (RoPE) [67] is widely adopted in modern LLMs [4, 23, 74]. It applies a position-dependent linear transformation to hidden representations, which in practice reduces to per-token matrix vector multiplications. Compiling this procedure into constraints directly entails proving a large number of position-dependent multiplications [70], which can introduce substantial overhead even with optimized matrix-multiplication proofs [72]. Finally, proving that the *output token* follows a prescribed decoding rule (e.g., argmax or sampling) requires expressing non-arithmetic control flow—comparisons, normalization, and distributional constraints—over a large vocabulary. Consequently, prior work may prove Transformer computation, yet still fall short of proving complete end-to-end inference.

Challenge 2: Proving long-form autoregressive generation efficiently. Even for a single query, an LLM typically produces a multi-token continuation through many autoregressive generation steps. It requires proving the correctness of an entire *sequence* of generation, rather than a single next-token generation as in prior zkLLM work [14, 33, 46, 58, 60, 70].

A straightforward strategy is to prove each token individually, but this quickly becomes impractical because the prover and verifier cost scales with the number of generated tokens. Even if recursive proof composition [18–20, 75] is used to compress proofs and reduce verifier overhead, it does not eliminate the dominant prover workload.

One might hope to leverage the fact that the prover observes the full transcript and instead generate a *batched* proof. However, existing batching techniques—including folding-based schemes [10, 40, 41]—still require prover complexity linear in the total computation being proved, i.e., the sum of all forward passes across steps. Crucially, it is not sufficient to prove only the final generation step: intermediate generations must also be correct and consistent, since a malicious server could otherwise manipulate intermediate steps to steer subsequent inputs while still producing a valid proof for the final token.

In agent executions this challenge is further amplified: the model is invoked repeatedly and each round appends new tokens to the running transcript, rapidly increasing the subsequent generation context length.

Challenge 3: Proving external tool interactions. Unlike pure LLM inference, an agent execution interleaves model-generated tokens with *externally injected* tokens from tools (e.g., web search, APIs, or sandboxes). These tokens are appended to the context and can directly steer subsequent generation and actions. Existing zkLLM systems that focus on standalone model inference [14, 58, 70] do not model such tool invocations, and therefore provide no integrity guarantees for (i) the tool outputs themselves, or (ii) their causal linkage to later model outputs. This leaves a critical gap: a malicious provider could tamper with tool results or fabricate plausible-looking tool traces while still producing a seemingly coherent final response. Closing this gap requires binding tool observations into the proof—including authenticating responses against the claimed endpoints, proving the correctness of local computation under deterministic semantics when applicable, and composing these guarantees with LLM-generation correctness into a single end-to-end statement.

1.1 Our Contributions

To address the above challenges, we present zkAgent, a zero-knowledge proof for end-to-end agent executions with tool use. Our contributions are as follows.

Complete inference proof. To bridge the gap between proofs that cover only the *internal Transformer block computation* and a *complete* inference, we design an end-to-end inference protocol that proves the entire pipeline from discrete input tokens to the output token. Concretely, our protocol covers the previously omitted components—token-to-embedding lookup, positional encoding, and decoding—in addition to the Transformer computation.

For token-to-embedding lookup, we observe that in common LLMs [4, 59, 74], the embedding matrix is highly rectangular: the vocabulary size is typically orders of magnitude larger than the embedding dimension. Leveraging this structure, we design an efficient preprocessing-based *matrix lookup* scheme that decouples the prover's complexity from the vocabulary size. Moreover, since embedding tables are fixed after training, we leverage a one-time preprocessing step to amortize their proving cost across all future inferences.

For positional encoding, we develop specialized protocols tailored to widely used positional encoding schemes. The

most challenging case is rotary positional encoding (RoPE). By exploiting RoPE's element-wise structure, we avoid the naïve compilation into many matrix multiplications [70] and reduce the overhead from $O(Td_h^2)$ to $O(Td_h)$, where T is the prompt length and d_h is the head dimension. For decoding, we prove that the published next-token choice is consistent with a specified decoding rule (e.g., greedy decoding), enforcing the required comparisons via efficient range-checks [63, 64].

Finally, for the Transformer computation, we build on the state-of-the-art zkGPT [58] framework and replace its lookup protocol [32, 64] with the more efficient Shout [63] lookup, substantially reducing the proving overhead.

One-shot transcript proving via masking. Proving an agent run by separately proving each token generation step-by-step leads to prohibitive prover cost. Therefore, we introduce a *one-shot transcript proving* technique that verifies the entire agent execution using a *single* masked forward pass proof.

A natural strawman approach is to prove the entire agent transcript at once: given the full sequence of tokens, the prover performs a single forward computation that outputs all tokens in parallel and proves consistency with the transcript. At first glance, this appears plausible, since the prover already possesses the complete transcript and all intermediate states.

However, this strawman is fundamentally problematic because LLM generation is autoregressive [9, 76]: for each position t, the logits used to produce $\mathtt{tr}_{t+1}$ must be a function only of the prefix $\mathtt{tr}_{\leq t}$, and the proof must explicitly enforce this causal dependence to rule out witnesses that implicitly depend on future tokens. A naive forward pass on the full transcript violates this principle, as earlier token generation may inadvertently depend on later tokens.

Our key observation is that this autoregressive constraint can be enforced via *causal attention mask* [9, 76] in every self-attention layer: for each position t, attention weights to positions $> t$ are forced to be zero. This guarantees that the logits at position t depend only on $\mathtt{tr}_{\leq t}$, yet all logits for $t = 0, \ldots, T-1$ can still be computed in parallel from the full transcript, enabling one-shot proving of the entire sequence. We formally prove that, under the causal attention mask, enforcing per-position decoding consistency at all marked positions is equivalent to sequential autoregressive generation of those tokens on the same transcript. We introduce a causal mask matrix and design the first protocol that avoids the *one-proof-per-token* structure of stepwise approaches and enable proving long, multi-inference agent traces via a single causally masked forward-pass proof.

Verifiable tool observations and execution. Modern agent executions rely on external tools in two categories: (i) remote network API calls (e.g., web search), and (ii) local code execution. To securely compose tool use into end-to-end agent proving, we design tool-specific protocols that bind each tool invocation and observation to cryptographic attestations.

For remote API calls, agent integrity requires that the prover faithfully use the responses returned by the intended web servers, rather than forged, censored, or selectively modified results. Similar challenges have been studied in blockchains when bridging Web2 and Web3 contents [81, 85]. Accordingly, we incorporate zkTLS-style attestations [81], which cryptographically bind each recorded response to its corresponding request under the identity of the target server. For code execution, we target the widely adopted "Python sandbox" used by most agent frameworks [2, 31, 50]. We build a toolchain that proves the correctness of the code execution using existing zkVM techniques, enforcing that the outputs match deterministic execution semantics. Beyond binding tool outputs, we also enforce the *well-formedness* of tool invocations. Concretely, we prove that any purported tool call in the transcript is a syntactically valid JSON object and conforms to a public tool-invocation schema [3, 35, 51], and that the subsequent tool observation is linked to that invocation. We realize this schema-compliance check via provable parsing techniques such as Coral [8]. Finally, to support transcripts that interleave model outputs with externally injected tool tokens, we augment the transcript with a *generation mask* indicating which positions are model-generated.

Prototype and evaluation. We implement zkAgent in C++. For agent transcript proving on GPT-2 with a 30-token prompt and final context length $T = 512$ (i.e., 482 generated tokens), zkAgent proves the entire transcript in one-shot with 508.46s, amortizing to 1.05 s per generated token, while verification and proof size remains succinct (0.51 s and 108MB). This yields a 294× reduction in prover time over the state of the art, zkGPT [58], with step-by-step generation (149,192.6 s), while also reducing verification time by 9,690× (0.45 s vs. 4361.09 s). Microbenchmarks further validate the efficiency of the additional components needed for *complete* inference beyond Transformer blocks: our preprocessing-based embedding *matrix* lookup incurs a one-time setup cost of 759.913 s and 4.52 GB auxiliary storage, and achieves 0.40 s online proving for 1,024 embedding queries; our RoPE encoding reduces proving time by 11.5×–31.3× versus a naïve matmul arithmetization [70]; and our Shout-based lookup backend reduces proving cost for decoding-related lookups by 10.7×–24.4×.

Finally, on realistic agent traces, zkAgent certifies end-to-end executions with practical overhead. For a 154-token weather-agent trace and a 146-token coding-assistant trace, total proving time is 238.06 s and 235.27 s, with succinct verification (0.55 s and 0.49 s) and proof sizes of 42.55 MB and 42.38 MB, respectively. In both cases, proving is dominated by the inference proof, while schema validation and tool-authenticity contribute only a small additional overhead.

2 Preliminaries

2.1 Notations

We use λ to denote the security parameter. Let $\mathbb{F}$ be a field, and let $\mathbb{F}$ be a prime field of order p such that $\log p = \Omega(\lambda)$.

For $n \in \mathbb{N}$, let $[n] := \{0, \ldots, n-1\}$. We use boldface to denote vectors and matrices, e.g., $\boldsymbol{x}$ and $\boldsymbol{W}$. Indices start from 0: for $\boldsymbol{x} \in \mathbb{F}^n$, its i-th entry is $\boldsymbol{x}_i$; for $\boldsymbol{W} \in \mathbb{F}^{m \times n}$, its entry is $\boldsymbol{W}_{i,j}$, its i-th row is $\boldsymbol{W}_{i,:} \in \mathbb{F}^n$, and its j-th column is $\boldsymbol{W}_{:,j} \in \mathbb{F}^m$. We use slice notation $\boldsymbol{x}[i:j] := (\boldsymbol{x}_i, \ldots, \boldsymbol{x}_{j-1})$. We also use $\boldsymbol{x}_{\leq n}$ to denote $\boldsymbol{x}[0:n]$. For $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{F}^n$, let $\boldsymbol{x} \circ \boldsymbol{y} := (\boldsymbol{x}_0 \boldsymbol{y}_0, \ldots, \boldsymbol{x}_{n-1} \boldsymbol{y}_{n-1})$ be the Hadamard product. For $I \subseteq [m]$ and $J \subseteq [n]$, let $\boldsymbol{W}_{I,J}$ be the submatrix of $\boldsymbol{W}$ restricted to rows I and columns J (and similarly $\boldsymbol{W}_{I,:}$ and $\boldsymbol{W}_{:,J}$). Given an index vector $\boldsymbol{x} = (x_1, \ldots, x_k)$ with $x_t \in [m]$, define the row-selected submatrix $\boldsymbol{W}(\boldsymbol{x}) := \boldsymbol{W}_{\boldsymbol{x},:} = (\boldsymbol{W}_{x_1,:}, \ldots, \boldsymbol{W}_{x_k,:})$. We use $\mathsf{tr} := (t_i)_{i \in [T]} \in [N]^T$ to denote the token-level agent's *transcript*, namely the chronological sequence of all context tokens fed to and produced by the agent throughout an execution. We use $\boldsymbol{H}$ to denote the hidden-state tensor during the LLM inference.

2.2 GKR Protocol for ML

The GKR protocol [30] is an interactive proof system designed to efficiently prove the correctness of computations over layered arithmetic circuits. Compared with other SNARKs that commit to the full execution trace (e.g., HyperPlonk [15] and Spartan [62]), GKR allows the prover to iteratively reduce the claim about one layer to claims about the preceding layer and answer the verifier's random-point queries on-the-fly, which typically avoids the high overhead of committing to the entire execution trace. Therefore, GKR is widely used in many zkML systems [45, 58, 70].

Since GKR only natively supports arithmetic constraints, we follow prior work and split the constraints arising in quantized LLM inference: (i) *arithmetic relations* dominated by additions and multiplications (e.g., matrix multiplications), and (ii) *non-arithmetic relations* such as activation functions and the exponentiation function. The former are handled naturally by GKR together with dedicated matmul subprotocols [45, 72], while the latter are typically handled via lookup arguments with additional consistency checks [64, 70].

Multilinear Extension. We define the set $\mathcal{F}_n^{(\leq d)}$ to be all n-variate polynomials $f : \mathbb{F}^n \to \mathbb{F}$ where the degree is at most d in each variable. In particular, an n-variate polynomial f is said to be *multilinear* if $f \in \mathcal{F}_n^{(\leq 1)}$. For any $f : \{0,1\}^n \to \mathbb{F}$, there is a unique multilinear polynomial $\tilde{f} : \mathbb{F}^n \to \mathbb{F}$ such that $\tilde{f}(\boldsymbol{x}) = f(\boldsymbol{x})$ for all $\boldsymbol{x} \in \{0,1\}^n$. The polynomial $\tilde{f}$ is called the *multilinear extension* (MLE) of f, and can be expressed as $\tilde{f}(\boldsymbol{X}) = \sum_{\boldsymbol{x} \in \{0,1\}^n} f(\boldsymbol{x}) \cdot \widetilde{eq}(\boldsymbol{x}, \boldsymbol{X})$, where $\widetilde{eq}(\boldsymbol{x}, \boldsymbol{X}) := \prod_{i=0}^{n-1} (\boldsymbol{x}_i \boldsymbol{X}_i + (1-\boldsymbol{x}_i)(1-\boldsymbol{X}_i))$.

Sumcheck Protocol. GKR uses the sumcheck protocol [47] as a core building block. At a high level, the sumcheck protocol enables the prover to convince the verifier that the sum of a polynomial $f : \mathbb{F}^n \to \mathbb{F}$ on the boolean hypercube equals the claimed value v, i.e. $v = \sum_{\boldsymbol{x} \in \{0,1\}^n} f(\boldsymbol{x})$. The formal construction of the sumcheck protocol is presented in Appendix B.1.

ML-friendly GKR variant. However, the classical GKR protocol is not suitable for ML scenarios: model parameters are usually committed in the input layer. Since in classical GKR, each gate only receives input from the preceding layer, these weights must be relayed across the circuit to reach the desired layer. This process introduces numerous additional gates, leading to high computational overhead. We therefore adopt the variant proposed by Zhang et al. in [87], where gate definitions are extended to allow inputs either from the preceding layer or directly from the input layer. The formal construction of ML-friendly GKR protocol is presented in Appendix B.2; we refer to [45, 87] for further details.

2.3 Lookup arguments

Lookup arguments [32] enable a prover to convince a verifier that a multiset of queried values appears in a committed lookup table, without explicitly arithmetizing the underlying operation. Concretely, given a committed table $\boldsymbol{b} \in \mathbb{F}^N$ and a committed query vector $\boldsymbol{a} \in \mathbb{F}^\ell$, a lookup argument proves the multiset inclusion $\{\boldsymbol{a}_i\}_{i \in [\ell]} \subseteq \{\boldsymbol{b}_j\}_{j \in [N]}$. This primitive is widely used in zkML [33, 58, 70] pipelines to implement non-linear functions (e.g., $\exp(\cdot)$) and enforce range checks, by reducing them to table-membership checks.

In practice, lookup arguments fall into three common regimes. *(i) Small tables.* Many workloads rely on small evaluation tables (e.g., 8–16 bit range tables) queried repeatedly; for this regime, protocols whose cost scales roughly with query size ℓ plus the table size N are effective [32, 63]. *(ii) Large but structured tables.* When a large table admits exploitable structure, one can often decompose it into smaller components so that the proving cost depends on the decomposed representation rather than the original size [64]. *(iii) Huge unstructured tables.* Finally, some applications require lookups into tables that are both massive and essentially unstructured (e.g., vocabulary-sized embedding matrices), where any *online* cost linear in N is prohibitive. This motivates *preprocessed* lookup arguments, which amortize all table-dependent work into a one-time preprocessing on the table, yielding an online prover cost that depends only on ℓ and is independent of N.

We briefly review the cached-quotients (cq) lookup argument [25]. At a high level, cq combines logarithmic-derivative reduction [32] with univariate polynomial commitments (e.g., KZG [38]) and a one-time preprocessing, enabling efficient table-side divisibility checks during the online phase. A detailed description of cq is provided in Appendix C.

2.4 LLM Inference Pipeline

We briefly review the inference pipeline of decoder-only LLMs at the granularity relevant to our proof. As illustrated in Fig. 1, given an input prompt, the system (i) tokenizes it into public token IDs, then repeatedly performs (ii) token-to-embedding lookup, (iii) positional encoding (e.g., APE is

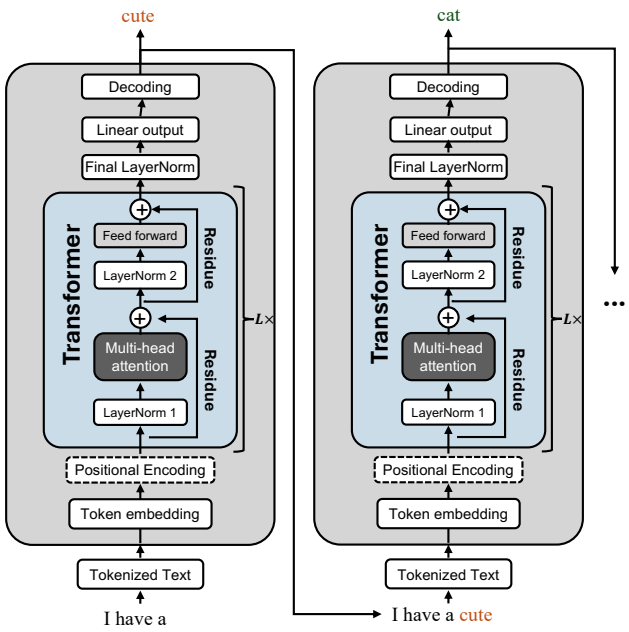

Figure 1: LLM Inference Pipeline

added to the input embeddings, RoPE is applied within the self-attention computation in each block), (iv) a stack of Transformer blocks, and (v) an output head plus a public decoding rule (e.g., greedy arg max) to select the next token. Step (vi) autoregressively appends the selected token to the context and repeats until termination. Since tokenization is public, we treat it as preprocessing and start the proved computation from the transcript token IDs; concrete layer equations are deferred to Appendix A.

2.5 Agent: Tool-Augmented LLM

An agent augments base LLMs with access to external tools (e.g., APIs or code execution) and orchestrates them in an interactive loop. Starting from the initial user input, the agent repeatedly (i) runs the LLM on the current conversation context, (ii) if the model emits a tool call, executes the corresponding tool(s), and (iii) appends the resulting tool observations back into the context before the next model invocation (Fig. 2).

We model the agent's execution trace as a growing *transcript*. Let $\mathcal{V}$ be a fixed vocabulary, and let $\mathtt{tr} = (t_0, \ldots, t_{n-1}) \in \mathcal{V}^n$ denote the token sequence recording the entire run. For presentation, it is convenient to view $\mathtt{tr}$ as an ordered concatenation of *segments*—including user messages, LLM outputs, tool-call spans, and tool-observation spans. At any time, the *context* provided to the LLM is simply the current prefix $\mathtt{tr}_{\leq k}$ of the transcript; each model generation step and each tool observation extends $\mathtt{tr}$.

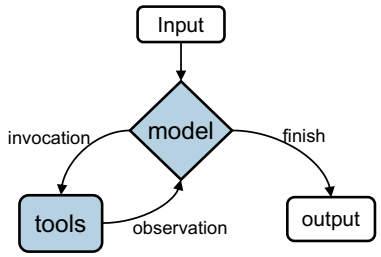

Figure 2: Agent loop

3 Complete Inference Proof for LLM

Most prior works [58, 70] primarily focus on optimizing proof generation for the Transformer computation while omitting other components that are indispensable in modern LLM inference pipelines, including token embedding, positional encodings, and the decoding procedure. In this section, we present formal constructions for these components to enable complete inference proofs.

3.1 Token-to-Embedding Lookup

Let $N := |\mathcal{V}|$ be the vocabulary size and d the embedding dimension. Given a token sequence in transcript $\mathtt{tr} = (t_i)_{i\in[T]} \in [N]^T$ and the private embedding table $\boldsymbol{W}_E \in \mathbb{F}^{N\times d}$, the prover must show that the activation matrix fed into subsequent computation, $\boldsymbol{H}_0 \in \mathbb{F}^{T\times d}$, is exactly the row-selected submatrix:

$$\forall i \in [T], \quad \boldsymbol{H}_0[i,:] \;=\; \boldsymbol{W}_E[t_i,:]. \tag{1}$$

While the lookup argument used in prior zkLLM schemes [58, 70] can in principle be extended to matrix lookup, it incurs prover overhead linear in the embedding table size Nd (about 2^{25} entries for GPT-2), far exceeding the number of queries (typically a few thousand). In light of this inefficiency, We present an optimized protocol based on $\mathtt{cq}$ [25] (see Appendix C), achieving online proving cost linear in T and d after one-time table-dependent preprocessing.

One-time preprocessing on the table side. A naïve reduction to d independent column lookups is vulnerable to *mix-and-match*[1]. We prevent this by binding each entire row with a single random linear combination. Let ρ be a random challenge derived via Fiat-Shamir. For each table index $u \in [N]$ and each query position $i \in [T]$, define folded scalars

$$b_u \;:=\; u \;+\; \textstyle\sum_{j\in[d]} \rho^{j+1} \cdot \boldsymbol{W}_E[u,j], \tag{2}$$

$$a_i \;:=\; t_i \;+\; \textstyle\sum_{j\in[d]} \rho^{j+1} \cdot \boldsymbol{H}_0[i,j]. \tag{3}$$

The constant terms u and t_i bind the claimed row token index, and the ρ^{j+1}-weighted sum binds all d entries in the row. It suffices to prove the multiset inclusion

$$\{a_i\}_{i\in[T]} \subseteq \{b_u\}_{u\in[N]}. \tag{4}$$

[1] A malicious prover might attempt to splice coordinates from different rows or columns to forge a queried embedding row.

Soundness follows from Schwartz-Zippel lemma: for any fixed i, if $\boldsymbol{H}_0[i,:] \neq \boldsymbol{W}_E[t_i,:]$, then $a_i = b_{t_i}$ would require a nonzero degree-$\leq d$ polynomial to vanish at ρ, which happens with probability at most $d/|\mathbb{F}|$.

A key obstacle is that the folding challenge ρ must be sampled online after the prover commits to $\boldsymbol{H}_0$; otherwise, a cheating prover can choose $\boldsymbol{H}_0[i,:] \neq \boldsymbol{W}_E[t_i,:]$ while still satisfy Eq. (3). Hence the folded table values $\{b_u\}$ depend on ρ and cannot be preprocessed as a fixed instance. Instead, we preprocess each table column separately and aggregate them online using the linearity of the KZG commitments [38].

Write the j-th embedding column as $\boldsymbol{w}_j := \boldsymbol{W}_E[:,j] \in \mathbb{F}^N$, and define the public index vector $\mathsf{idx} := [N] \in \mathbb{F}^N$. Let $\mathsf{Preproc}(\cdot)$ denote the preprocessing procedure of the lookup argument $\mathfrak{cq}$ [25] (Appendix C), and let pp denote its output. Concretely, for an input vector $\boldsymbol{b} \in \mathbb{F}^N$ with interpolant $T(X) \in \mathbb{F}_{<N}[X]$ over the domain V, $\mathsf{Preproc}(\boldsymbol{b})$ outputs: (i) commitments to the Lagrange basis $\{\mathsf{Commit}(L_j)\}_{j\in[N]}$ on V; (ii) auxiliary commitments $\{\mathsf{Commit}(\Delta L_j = \frac{L_j(X)-L(0)}{X})\}_{j\in[N]}$ for efficient openings at $X=0$; (iii) the table commitment $c_T := \mathsf{Commit}(T)$; and (iv) cached-quotient commitments $\{c_{Q,j}\}_{j\in[N]}$ for polynomials Q_j satisfying $L_j(X)T(X) = b_jL_j(X) + Z_V(X)Q_j(X)$, where $Z_V(X) := \prod_{x\in V}(X-x)$ is a vanishing polynomial. All these objects are $\mathbb{F}$-linear and thus can be aggregated by linear combination. So in a one-time setup for a fixed $\boldsymbol{W}_E$, compute

$$\mathsf{pp}_{\mathsf{idx}} \leftarrow \mathsf{Preproc}(\mathsf{idx}), \tag{5}$$

$$\mathsf{pp}_{E,j} \leftarrow \mathsf{Preproc}(\boldsymbol{w}_j), \qquad \text{for } j \in [d]. \tag{6}$$

After sampling ρ online, we can fold them:

$$\mathsf{pp}_\rho \;:=\; \mathsf{pp}_{\mathsf{idx}} + \textstyle\sum_{j\in[d]} \rho^{j+1}\, \mathsf{pp}_{E,j}. \tag{7}$$

By the $\mathbb{F}$-linearity of all preprocessed objects (e.g., c_T, $\{\mathsf{Commit}(L_j)\}$, $\{\mathsf{Commit}(\Delta L_j)\}$, and $\{c_{Q,j}\}$), the prover need not materialize pp_ρ: it combines only the components needed for the queried rows. Then we run the table side of $\mathfrak{cq}$, which consists of two checks.

Sample $\beta \xleftarrow{\$} \mathbb{F}$ and define the logarithmic-derivative witnesses

$$f_u \;:=\; \frac{m_u}{b_u+\beta} \quad (u \in [N]), \tag{8}$$

where $\boldsymbol{m} := \{m_u\}_{u\in[N]} \in \mathbb{F}^N$ is the multiplicity of b_u in the query multiset $\{a_i\}$. Let H be the size-N evaluation domain used by the univariate commitment and let $B(X), M(X), F(X) \in \mathbb{F}_{<N}[X]$ interpolate $(b_u), (m_u), (f_u)$ over H. The pointwise relation $f_u(b_u+\beta) = m_u$ for all u is equivalent to the divisibility constraint

$$\big(B(X)+\beta\big)\cdot F(X) - M(X) \;=\; Q(X)\cdot Z_H(X), \tag{9}$$

for some quotient polynomial $Q(X)$. Using pp_ρ, the prover proves Eq. (9) via $\mathfrak{cq}$'s pairing check. By sparsity, the required commitments are formed by combining only the basis/quotient terms indexed by $\mathsf{supp}(\boldsymbol{m})$, so the online cost scales with $|\mathsf{supp}(\boldsymbol{m})| \leq T$, not N.

The table side also outputs the sumcheck claim

$$z \;:=\; \sum_{u\in[N]} f_u \;=\; \sum_{x\in H} F(x). \tag{10}$$

For the standard choice of H as a multiplicative subgroup and $\deg(F) < N$, $\sum_{x\in H} F(x) = N\cdot F(0)$. Hence the verifier obtains z from a single opening of F at $X=0$ and sets $z \leftarrow N\cdot f_0$.

Multilinear online proving on the query side. Most preprocessed lookup arguments [25, 56, 83] exploit KZG linearity (and pairing-based verification) to support one-time preprocessing, and are thus most instantiated phrased over univariate polynomials. In contrast, the downstream GKR proof operates over multilinear extensions (MLEs); committing to the query side univariately would be incompatible with GKR and would introduce quasi-linear online overhead from FFT/interpolation. Following [12], we keep the table side univariate with preprocessing, and prove the query side in the multilinear world, reusing the commitment to $\boldsymbol{H}_0$.

Let $k_T := \lceil \log T \rceil$ and $k_d := \lceil \log d \rceil$. Interpret $c_0 := \mathsf{Commit}(\boldsymbol{H}_0)$ as a commitment to the MLE $\widetilde{H}_0 : \mathbb{F}^{k_T+k_d} \to \mathbb{F}$ such that

$$\widetilde{H}_0(\iota_T(i), \iota_d(j)) \;=\; \boldsymbol{H}_0[i,j] \qquad \text{for } (i,j) \in [T]\times[d],$$

where $\iota_T : [T] \to \{0,1\}^{k_T}$ and $\iota_d : [d] \to \{0,1\}^{k_d}$ are the standard boolean encodings with padding. From the transcript, the verifier derives MLEs $\widetilde{t} : \mathbb{F}^{k_T} \to \mathbb{F}$ and $\widetilde{w}_\rho : \mathbb{F}^{k_d} \to \mathbb{F}$, such that $\widetilde{t}(\iota_T(i)) = t_i$ and $\widetilde{w}_\rho(\iota_d(j)) = \rho^{j+1}$ (0 on padded indices).

Define the row-folded MLE

$$\widetilde{s}_\rho(x) \;:=\; \sum_{y\in\{0,1\}^{k_d}} \widetilde{w}_\rho(y)\cdot \widetilde{H}_0(x,y), \quad x \in \mathbb{F}^{k_T}, \tag{11}$$

and the folded query MLE

$$\widetilde{a}_\rho(x) \;:=\; \widetilde{t}(x) + \widetilde{s}_\rho(x). \tag{12}$$

By construction, we have $\widetilde{a}_\rho(\iota_T(i)) = a_i$ as in Eq. (3).

Let β be the Fiat–Shamir challenge sampled in the lookup reduction, and let $z \in \mathbb{F}$ be the sum claimed by the table-side proof. It remains to prove $z = \sum_{i\in[T]} 1/(a_i+\beta)$. Define $u_i := 1/(a_i+\beta)$ and let $\widetilde{U} : \mathbb{F}^{k_T} \to \mathbb{F}$ be the MLE with $\widetilde{U}(\iota_T(i)) = u_i$; the prover commits to $\widetilde{U}$.

We verify the query side using two constraints:

$$z = \sum_{x\in\{0,1\}^{k_T}} \widetilde{U}(x), \qquad \widetilde{U}(x)\cdot(\widetilde{a}_\rho(x)+\beta) = 1 \;\text{ on } \{0,1\}^{k_T}.$$

We combine them via a single batched sumcheck. Sample $\alpha \xleftarrow{\$} \mathbb{F}$ and $\gamma \xleftarrow{\$} \mathbb{F}^{k_T}$, and prove

$$\sum_{x\in\{0,1\}^{k_T}} \Big(\alpha\cdot\widetilde{U}(x) + \widetilde{eq}(\gamma,x)\cdot\big(\widetilde{U}(x)\cdot(\widetilde{a}_\rho(x)+\beta)-1\big)\Big) = \alpha\cdot z \tag{13}$$

Since $\widetilde{U}(x)$ appears in both terms, we can reuse the same bookkeeping table. Finally, sumcheck reduces verification of

Eq. (13) to polynomial openings at a random point $r \in \mathbb{F}^{k_T}$. To evaluate $\widetilde{a}_{\rho}(r)$, the verifier computes $\widetilde{t}(r)$ directly, and obtains $\widetilde{s}_{\rho}(r)$ via a standard inner sumcheck over $\{0,1\}^{k_d}$ for $\sum_y \widetilde{w}_{\rho}(y)\widetilde{H}_0(r,y)$, which reduces to an opening of $\widetilde{H}_0$ at a random point. Using these values, the verifier derives $\widetilde{a}_{\rho}(r)$ and checks the final-round identity implied by Eq. (13).

Cost Analysis. The one-time preprocessing cost is $O(dN\log N)$ field/group operations via optimization from [26, 88] and is amortized across inferences for a fixed model. Online, the additional prover work is dominated by: (i) $O(Td)$ field operations to form the folded query values a_i in Eq. (3), and (ii) $O(Td)$ group operations to produce the quotient commitment for the table-side pairing check. The verifier work is only an opening for $\boldsymbol{H}_0$ which can be batched, plus $O(\log T + \log d)$ field operations for sumcheck Eq. (11) and Eq. (13) and an evaluation to $\widetilde{t}(r)$. Crucially, the query-side proof composes directly with the MLE commitment to $\boldsymbol{H}_0$ already used by the downstream inference proof.

3.2 Efficient Proof for Positional Encoding

Positional encodings inject token-order information into decoder-only Transformers. We cover the two common variants (APE and RoPE) with acceptable overhead.

Absolute positional embeddings (APE). For APE-style models (e.g., GPT-2), the model contains a learned positional matrix $\boldsymbol{P} \in \mathbb{F}^{T_{\max}\times d}$, which committed as part of the private weights. Given the token embeddings $\boldsymbol{H}_0 \in \mathbb{F}^{T\times d}$, the positional-augmented activations are

$$\boldsymbol{H}_0^{\mathsf{pe}} := \boldsymbol{H}_0 + \boldsymbol{P}[0{:}T,:].$$

This is a purely linear relation between committed tensors, hence it can be natively handled by the downstream zkLLM backend (i.e., GKR circuit), incurring minor extra cost.

Rotary positional embeddings (RoPE). RoPE-style models (e.g., Llama [74]/Qwen [4]) inject position information by rotating the per-head Query/Key vectors inside each attention block. Fix one attention head and let d_h denote the head dimension (w.l.o.g. assume d_h is even).

After the usual linear projections and head reshaping, we obtain $\boldsymbol{Q},\boldsymbol{K} \in \mathbb{F}^{T\times d_h}$. Write $\boldsymbol{Q} = (\boldsymbol{q}_0^\top;\ldots;\boldsymbol{q}_{T-1}^\top)$ and $\boldsymbol{K} = (\boldsymbol{k}_0^\top;\ldots;\boldsymbol{k}_{T-1}^\top)$, where $\boldsymbol{q}_m,\boldsymbol{k}_m \in \mathbb{F}^{d_h}$ are the row vectors at position $m \in [T]$. RoPE applies a block-diagonal 2×2 rotation to each consecutive coordinate pair (Appendix A). Rather than treating them as matrix vector multiplications [70] with $O(Td_h^2)$ overhead, we use a substitute Hadamard with a permutation form. Define, for any row vector $\boldsymbol{u} = (u_0,\ldots,u_{d_h-1})$,

$$\pi(\boldsymbol{u}) := (-u_1, u_0, -u_3, u_2, \ldots, -u_{d_h-1}, u_{d_h-2}).$$

For each position $m \in [T]$, let $\boldsymbol{c}_m, \boldsymbol{s}_m \in \mathbb{F}^{d_h}$ be the quantized cosine/sine coefficient vectors that repeat each frequency twice:

$$\boldsymbol{c}_m = (\cos(m\theta_0),\cos(m\theta_0),\cos(m\theta_1),\cos(m\theta_1),\ldots),$$
$$\boldsymbol{s}_m = (\sin(m\theta_0),\sin(m\theta_0),\sin(m\theta_1),\sin(m\theta_1),\ldots),$$

where the frequencies $\{\theta_i\}_{i\in[d_h/2]}$ are public and fixed by the model architecture (e.g., $\theta_i = 10000^{-2i/d}$). Then RoPE can be written entrywise as

$$\boldsymbol{q}_m^{\mathsf{rot}} = \boldsymbol{q}_m \odot \boldsymbol{c}_m + \pi(\boldsymbol{q}_m)\odot \boldsymbol{s}_m,\ \boldsymbol{k}_m^{\mathsf{rot}} = \boldsymbol{k}_m \odot \boldsymbol{c}_m + \pi(\boldsymbol{k}_m)\odot \boldsymbol{s}_m.$$

Stacking all positions yields the batched form

$$\boldsymbol{Q}^{\mathsf{rot}} = \boldsymbol{Q}\odot\boldsymbol{C} + \pi(\boldsymbol{Q})\odot\boldsymbol{S}, \quad \boldsymbol{K}^{\mathsf{rot}} = \boldsymbol{K}\odot\boldsymbol{C} + \pi(\boldsymbol{K})\odot\boldsymbol{S},$$

where $\boldsymbol{C},\boldsymbol{S} \in \mathbb{F}^{T\times d_h}$ stack $\{\boldsymbol{c}_m\}_m$ and $\{\boldsymbol{s}_m\}_m$, and $\pi(\cdot)$ is applied row-wise.

Circuit realization and cost. With the above rewriting, RoPE contains only: (i) fixed wiring for the permutation $\pi(\cdot)$, (ii) linear sign flips and additions, and (iii) Hadamard products. These operations are directly supported by the GKR-style arithmetization used by our zkML backend, so RoPE integrates into the main circuit without introducing any dedicated subprotocol [72]. Moreover, the trigonometric tables $\boldsymbol{C},\boldsymbol{S}$ depend only on public positions and fixed frequencies. Under the agreed quantization, we treat $\boldsymbol{C},\boldsymbol{S}$ as public constants and exclude their generation from online proving. Consequently, the only additional multiplicative work is the two Hadamard products per tensor (Query and Key), i.e., overall $O(Td_h)$ additional field multiplications per head instead of $O(Td_h^2)$.

3.3 Efficient Proof for the Decoding Procedure

We prove correctness of the *decoding* step that maps the final hidden state to the next token. Fix one autoregressive step and let $N := |\mathcal{V}|$. Let $\boldsymbol{\ell} = (\ell_0,\ldots,\ell_{N-1}) \in \mathbb{F}^N$ be the quantized logits produced by the output projection, and let $y \in [N]$ be the output token id appended to the transcript.

Comparisons. We interpret field elements as signed integers under the agreed quantization, e.g., each ℓ_i represents an integer in $[-B,B)$ for a public bound $B \ll |\mathbb{F}|$. All inequalities are enforced via standard *range checks* on signed differences (e.g., via Shout [63]), so that comparisons are sound in the integer semantics rather than modulo $|\mathbb{F}|$.

Greedy decoding. For greedy decoding, correctness means $\ell_y \geq \ell_i$ for all $i \in [N]$. The prover introduces differences

$$\Delta_i := \ell_y - \ell_i \qquad (i \in [N]),$$

as well as the index y, and proves the constraints

$$\Delta_y = 0, \quad \Delta_i \in [0,2B) \quad \forall i \in [N]$$

Since $\Delta_i = \ell_y - \ell_i$, $\Delta_i \geq 0$ implies $\ell_y \geq \ell_i$ for all i, i.e., $y \in \arg\max_i \ell_i$.

Candidate-set decoding (top-k / nucleus). For sampling-based decoding, we separate (a) proving the candidate set and (b) proving Softmax and sampling inside it.

Top-k. The prover fixes a candidate set $S \subseteq [N]$ with $|S| = k$ and a threshold

$$\lambda := \min_{u\in S} \ell_u.$$

To express "in S" vs "out of S" without explicitly iterating over $[N] \setminus S$, we use a boolean selector vector $\boldsymbol{\chi} \in \{0,1\}^N$ where $\chi_i = 1 \iff i \in S$. We enforce:

$$\sum_{i\in[N]} \chi_i \;=\; k, \quad \chi_i(\chi_i - 1) = 0 \quad \forall i \in [N], \quad \chi_y = 1.$$

To certify the threshold semantics, the prover additionally provides an index $i_\star \in [N]$ such that $\chi_{i_\star} = 1$ and enforces $\lambda = \ell_{i_\star}$. Then we range-check the *masked* inequalities

$$\chi_i \cdot (\ell_i - \lambda) \in [0, 2B), \quad (1-\chi_i)\cdot(\lambda - \ell_i) \in [0, 2B) \quad \forall i \in [N].$$

Intuitively, selected indices must satisfy $\ell_i \geq \lambda$, and unselected indices must satisfy $\ell_i \leq \lambda$. In particular, any index with $\ell_i > \lambda$ must be selected, hence S is a valid top-k set.

Top-p (nucleus). Nucleus sampling [34] selects the smallest set whose Softmax mass exceeds p. We use the common cutoff characterization on probabilities: after computing probabilities $(p_i)_i$ (scaled by public θ), the prover supplies a cutoff $\lambda := \min_{u\in S} p_u$ and a selector vector $\boldsymbol{\chi}$ for S, and enforces

$$\chi_i \cdot (p_i - \lambda) \in [0, \theta], \quad (1-\chi_i)\cdot(\lambda - p_i) \in [0, \theta] \quad \forall i \in [N],$$

together with the two mass constraints

$$\sum_{u\in S} p_u \;\geq\; p\cdot\theta, \qquad \sum_{u\in S} p_u - \lambda \;\leq\; p\cdot\theta,$$

and $\chi_y = 1$. As above, range checks are applied to signed differences under the integer interpretation.

Softmax on the candidate set. Let S be the candidate set (either $S = [N]$ for full Softmax under top-p, or $|S| \ll N$ under top-k). For temperature $\tau > 0$, define scaled scores $s_i := \ell_i/\tau$ and probabilities

$$p_i \;:=\; \theta \cdot \frac{\exp(s_i)}{\sum_{j\in S}\exp(s_j)} \qquad (i \in S),$$

where θ is a public scaling factor for integer probabilities.

We avoid division by proving the multiplicative form. Let $e_i := \exp(s_i)$ and $D := \sum_{j\in S} e_j$. We prove

$$p_i \cdot D \;=\; \theta \cdot e_i \qquad \forall i \in S, \quad \text{and} \quad \sum_{i\in S} p_i \approx \theta,$$

where the normalization allows a small public rounding slack, depending on the quantization.

To prove exponentiation efficiently, we use the trick from zkGPT [58]: let $s_{\max} := \max_{i\in S} s_i$ and define $\delta_i := s_{\max} - s_i \in [0, B_{\mathrm{sm}}]$, so that $e_i = \exp(s_i - s_{\max}) = \exp(-\delta_i)$ is bounded. We decompose δ_i in base b, $\delta_i = \sum_{k=0}^{K-1} b^k x_i^{(k)}$ with digits $x_i^{(k)} \in [0, b-1]$, prove digit well-formedness by range checks, and use lookup tables for each k to obtain $y_i^{(k)} \approx \exp(-b^k x_i^{(k)})$. Finally we enforce $e_i = \prod_{k=0}^{K-1} y_i^{(k)}$. This reduces exponentiation to a few lookups plus a short product chain.

Cost. The decoding procedure adds $O(N)$ range checks on logit differences. For sampling-based decoding, the additional overhead is proving exponentiation. Top-k selects a candidate set S by logit ordering and proves Softmax only over S, whereas top-p must additionally certify the nucleus condition using normalized probabilities, requiring full Softmax.

4 zkAgent: One-Shot Verifiable Agent Execution

In this section, we lift our complete next-token generation proof to *agent executions* that interleave model generation with externally injected tokens (e.g., tool observations). Our goal is a complete proof that (i) proves all model-generated tokens in the transcript under a committed model and public decoding rule, and (ii) binds every injected tool observation to an authenticated execution of the claimed tool.

Threat model and scope. We assume an untrusted service provider that may deviate arbitrarily from the prescribed agent procedure, including substituting the model, fabricating transcripts, or tampering with tool observations. Our goal is *execution integrity*: the verifier should be convinced that the published transcript is consistent with the committed model, decoding rule, and specified tool interfaces. We do *not* aim to prove the correctness of external services; for network APIs we prove only *provenance* under a designated server identity.

Security statement. Given a commitment c_W to model parameters W, zkAgent proves: (i) *model integrity*: every transition marked as model-generated satisfies the decoding constraint in Eq. (17); (ii) *tool-call integrity*: every tool-call span is well-formed under a public schema and parses to a unique structured request; (iii) *tool integrity*: every tool observation in the transcript is the authentic output of executing the claimed tool on that request; and (iv) *binding*: the authenticated tool output is embedded into the transcript *exactly* as the token span the model conditions on.

4.1 One-shot Transcript Proving via Masking

An agent execution may contain many LLM invocations across a long-running transcript. Proving each token generation step-by-step is prohibitively expensive because the prover cost scales with the *sum* of all forward passes across the run. Instead, we batch *all* model-generated transitions into a *single* proof by proving one teacher-forced forward pass on the full transcript, while explicitly enforcing the autoregressive semantics via a causal attention mask.

To the best of our knowledge, our work is the first to enable *batched inference proof* by proving a single masked forward pass of a causal Transformer. Prior zkLLM approaches [58, 70, 80] either rely on step-by-step inference proofs or implicitly assume that batched inference proofs are possible without addressing the underlying caveat introduced by history-dependent generation. In particular, existing

works [80] discuss batched inference proofs only at a high level, without identifying the semantic mismatch between sequential and batched generation or presenting any concrete protocol for batching inference proofs. Beyond batching single continuation proving, we further provide a scheme that supports *multi-inference agent-trace proofs*, which prior works do not address. We also provide a complete construction and formally prove its correctness (see Appendix D).

Proving full continuation in one forward pass. Fix a public prompt prefix $\mathsf{tr}_{\mathsf{in}} = (t_0, \ldots, t_{T_0-1}) \in [N]^{T_0}$ and a claimed full transcript $\mathsf{tr} = (t_0, \ldots, t_{T-1}) \in [N]^T$ with $T > T_0$. Let $\mathsf{LLM}_W(\cdot)$ denote the *model evaluation* under weights W that outputs logits, and let $\mathsf{Dec} : \mathbb{F}^N \to [N]$ be the agreed decoding rule.

The intended online execution generates tokens sequentially: for each $i \in \{T_0 - 1, \ldots, T-2\}$, the server runs LLM_W on the prefix $\mathsf{tr}_{\le i} = (t_0, \ldots, t_i)$ to obtain the logit vector $\boldsymbol{\ell}_i \in \mathbb{F}^N$ and outputs $t_{i+1} = \mathsf{Dec}(\boldsymbol{\ell}_i)$. During online generation, the server must perform a *sequence* of forward passes because of autoregressive causality: the next token t_{i+1} depends on the model evaluated on the *current* prefix $\mathsf{tr}_{\le i}$. As the prefix grows by one token at a time, each step requires re-evaluating LLM_W on a longer context.

During proving, however, the prover already knows the entire transcript tr. This eliminates the need to *reconstruct* the transcript token-by-token via a sequence of growing-prefix forward passes. Instead, the prover can evaluate LLM_W *once* on the full length-T sequence, feeding the ground-truth token t_i at every position as input. This yields all per-position logits for next-token prediction. Then, we can adopt a *one-step shift* constraint that the logit vector at position i predicts the next token t_{i+1}. Concretely, letting $L := \mathsf{LLM}_W(\mathsf{tr}) \in \mathbb{F}^{T \times N}$ and $\boldsymbol{\ell}_i := L[i,:]$, *shift consistency* requires that

$$t_{i+1} \;=\; \mathsf{Dec}(\boldsymbol{\ell}_i) \quad \text{for all } i \in \{T_0 - 1, \ldots, T-2\}, \tag{14}$$

while the last row $\boldsymbol{\ell}_{T-1}$ is unused.

The only subtlety is *autoregressive causality*: in a naïve full-sequence evaluation, the representation at position i could depend on future tokens $t_{>i}$. In a decoder-only Transformer, self-attention is the only sublayer that mixes information across positions; all other sublayers (LayerNorm, MLP, and residual connections) are position-wise and cannot introduce future dependence. Therefore, it suffices to enforce causality in every self-attention layer so that position i can attend only to positions $\le i$. We enforce this by applying a *causal attention mask* (i.e., masked self-attention) in every attention layer.

Concretely, for a single head with query/key/value matrices $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{F}^{T \times d_h}$, define

$$\mathsf{Attn}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) \;=\; \mathsf{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top + \boldsymbol{M}^{\mathsf{cau}}}{\sqrt{d_h}} \right) \boldsymbol{V}, \tag{15}$$

where $\boldsymbol{M}^{\mathsf{cau}}_{ij} = 0$ for $j \le i$ and $\boldsymbol{M}^{\mathsf{cau}}_{ij} = -\infty$ otherwise. Equivalently, writing $s_{ij} := \langle q_i, k_j \rangle / \sqrt{d_h}$, the masked softmax

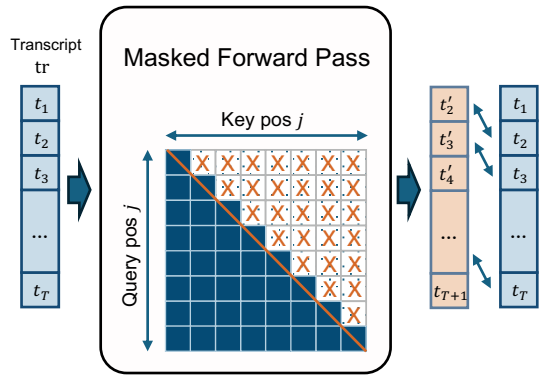

Figure 3: One-shot Transcript Proving via Masking

weights satisfy

$$\alpha_{ij} = \frac{\exp(s_{ij})}{\sum_{k \le i} \exp(s_{ik})} \quad (j \le i), \qquad \alpha_{ij} = 0 \;\; \text{(otherwise)}. \tag{16}$$

Eq. (16) makes the causality guarantee explicit: at position i, both the weighted sum and the softmax normalization range only over indices $j \le i$. Hence, no future token $t_{>i}$ can affect the hidden state (and thus logits) at position i. As a result, the logit $\boldsymbol{\ell}_i$ computed in the length-T masked self-attention run coincides with the logit obtained by an online run on the prefix $\mathsf{tr}_{\le i}$. We formalize and prove this *prefix equivalence* property in Appendix D. We want to clarify this evaluation mode is standard in supervised fine-tuning of LLMs [53] (often referred to as *teacher forcing*), where providing the ground-truth sequence enables computing next-token logits for all positions in parallel, substantially improving efficiency.

In arithmetic circuits, $-\infty$ is not representable. We therefore implement Eq. (16) *by construction*: for each query position i, we materialize only scores $\{s_{ij}\}_{j \le i}$, and define $\alpha_{ij} = 0$ for all $j > i$, so masked entries never enter the softmax denominator. This realizes the restricted-softmax semantics exactly and reduces exponentiation lookups from T^2 to $T(T+1)/2$.

4.2 Tool binding for verifiable agent execution

Agents interleave model-generated tokens with *tool observations* that are injected into the context and can steer subsequent decoding. Transcript-level model integrity alone is insufficient in this setting: a malicious prover could fabricate tool outputs while still producing a valid transcript. We therefore require *tool integrity and binding*: every injected observation must be cryptographically tied to an authenticated execution of the claimed tool on the request parsed from the transcript.

From single continuations to interleaved agent traces. First, we enable the transcript to interleave model-generated tokens with external injected tokens by introducing a generation mask $\boldsymbol{g} = (g_0, \ldots, g_{T-2}) \in \{0,1\}^{T-1}$ indicating whether

the next token is model-generated:

$$g_i = 1 \iff t_{i+1} \text{ is produced by } \mathsf{LLM}_W \text{ from prefix } \mathsf{tr}_{\le i},$$
$$g_i = 0 \iff t_{i+1} \text{ is injected externally.}$$

Given a teacher-forced forward pass on tr that yields logits $(\boldsymbol{\ell}_0, \ldots, \boldsymbol{\ell}_{T-1})$, we enforce shifted constraints only on model-generated transitions:

$$\forall i \in \{0, \ldots, T-2\}, \quad g_i = 1 \Rightarrow t_{i+1} = \mathsf{Dec}(\boldsymbol{\ell}_i), \tag{17}$$

while leaving injected tokens unconstrained with $g_i = 0$.

A crucial requirement is that $\boldsymbol{g}$ is *derived from the transcript structure*, not chosen by the prover (otherwise the prover could set $\boldsymbol{g} = \mathbf{0}$ and satisfy Eq. (17) vacuously). When the full transcript tr is public, $\boldsymbol{g}$ is uniquely determined by a public, unambiguous transcript grammar. When deployments hide internal reasoning tokens (e.g., chain-of-thought) [16], we distinguish tr from a *public transcript* $\mathsf{tr}^{\mathsf{pub}}$ containing all user/tool tokens and only the model-visible outputs. Let R denote the hidden span; the provider publishes a binding commitment $c_R := \mathsf{Com}(R)$ and length $L_R := |R|$. Given $(\mathsf{tr}^{\mathsf{pub}}, c_R, L_R)$ and the grammar, the verifier reconstructs the unique segmentation and hence $\boldsymbol{g}$. Any attempt to substitute R while keeping the same public behavior would alter later verified model outputs and violate Eq. (17); if it does not, the substitution is observationally irrelevant to the verified transcript.

To rule out *early termination*, we require in the grammar that every model-generated span ends with a designated termination (e.g., EOS/EOT) that is itself model-generated; premature truncation then either fails parsing or fails Eq. (17).

Tool-call parsing and binding. Each tool step consists of (1) a *tool-call span* (model-generated) that deterministically parses to a structured request req_i, and (2) a *tool-observation span* (externally injected). To ensure that the tool-call schema is correctly followed, we leverage *Coral* [8], a system for verifying a committed stream corresponds to a structured object. Coral guarantees that each tool-call span conforms to the expected structure and that all necessary parameters are present and valid. We bind tools via *plaintext equality*: the observation span in tr must equal the cleartext token sequence of the authenticated response object resp_i opened from the tool subproof. Thus, subsequent verified generation is guaranteed to condition on the same observation whose provenance/execution is proven.

Authenticated API tools via zkTLS. For networked tools (e.g., web search and REST/RPC APIs), the main requirement is authenticity of the returned response under a designated server endpoint. We leverage zkTLS-style techniques [73, 81] to attest to the contents of a TLS session. For invocation i with server identity id_i, request req_i, and response resp_i, the prover produces an authenticity proof π_i^{io} such that

$$\mathsf{Verify}_{\mathsf{io}}(\mathsf{id}_i, \mathsf{req}_i, \mathsf{resp}_i, \pi_i^{\mathsf{io}}) = 1.$$

and we bind resp_i into tr via plaintext equality as above.

Deterministic computation tools via Depyler and zkVM. For deterministic sandbox tools, we avoid proving a full Python interpreter by transpiling the model-produced script $\mathsf{program}_i$ to Rust $\mathsf{Rust}(\mathsf{program}_i)$ using Depyler [54] [2], compiling to RISC-V, and proving execution in a zkVM (e.g., SP1 [68, 69]). The prover produces π_i^{vm} such that

$$\mathsf{Verify}_{\mathsf{vm}}(\mathsf{Rust}(\mathsf{program}_i), \mathsf{output}_i, \pi_i^{\mathsf{vm}}) = 1,$$

and binds the injected observation by enforcing that the corresponding transcript slice equals the cleartext tokens of output_i. We require $\mathsf{program}_i$ and $\mathsf{Rust}(\mathsf{program}_i)$ to be public for independent verification, as directly proving the transpilation process is prohibitive.

Composition with one-shot transcript proving. Finally, we compose the tool-related subproofs above with the one-shot masked transcript proof (Sec. 4.1). The resulting end-to-end proof guarantees (i) that all model-generated tokens are consistent with the committed model and decoding rule, and (ii) that every externally injected tool observation is both authenticated by its tool subproof and bound to the exact span.

5 Evaluation

In this section, we show the implementation of our one-shot LLM proof system, as well as benchmark results compared with the best of previous work [14, 58, 70]. We also present the performance of our verifiable agent execution.

5.1 Implementation

Software. Our work is mainly implemented in C++. Our sumcheck and ML-friendly GKR implementations are adapted from zkGPT codebase [58]. For our preprocessing-based matrix lookup, we instantiate the underlying univariate polynomial commitment with KZG [38]. We adopt Shout [63] for lookup and range checks. We use KZH [37] for multilinear commitments. We use the `mcl` library[3] for finite-field arithmetic and elliptic-curve operations over BN254.

Hardware. All experiments are conducted on a server with an Intel Xeon 6982P-C (3.20 GHz, 32 cores) and 512 GB of RAM. We leverage 32-thread parallelization by default.

Quantization. Following prior work [45, 58], we represent each floating-point tensor using signed integers together with a *public* scaling factor. Concretely, a real value x is encoded as an integer $q \in \mathbb{Z}$ such that $x \approx q \cdot s$, where we choose a scale of the form $s = c \cdot 2^e$. We evaluate the impact of quantization on both perplexity and end-to-end agent behavior in Appendix E.

Baselines. We compare against representative prior open-sourced ZK frameworks for LLM inference, including

[2] Depyler can preserve the semantics of the original script, so $\mathsf{Rust}(\mathsf{program}_i)$ implements the same computation as $\mathsf{program}_i$.

[3] https://github.com/herumi/mcl

ZKML [14], zkGPT [58], and zkLLM [70]. We do not evaluate the protocol by Lu *et al.* [46] because it is not open-sourced. We also exclude Hao *et al.* [33]: their released code only implements a subset of nonlinear layers, does not provide an end-to-end implementation, and the reported proving cost is prohibitively high. zkPytorch [80] does not provide any concrete implementation or empirical evaluation.

All included open-source baselines natively support only fixed prompt lengths, with prover cost scaling at least linearly in context length. To enable an apples-to-apples comparison across varying context lengths, we extend them to prompt-length–parametric setting. For ZKML [14], the public repository contains a GPT-2 example based on a distilgpt2 TFLite model with fixed 64-token prompt length, and its converter and backend do not implement GeLU/ERF. We therefore add GeLU support and build a prompt-length–parametric compiler targeting their backend. For zkGPT [58], the released implementation provides a customized GKR circuit specialized to a 30-token GPT-2 configuration; we generalize it into a prompt-length–parametric circuit generator matching our GPT-2 evaluation setting. zkLLM [70] does not provide a GPT-2 instantiation; when needed, we use the corresponding results reported in zkGPT [58]. We release our refined baseline implementations as an additional part of our artifact.

For token-to-embedding lookup, we compare our preprocessing-based *matrix* lookup with general lookup scheme [32] used in zkGPT [58] and all prior matrix lookup protocol, including Segment Lookup [21], CosetLkup [24], and Campanelli *et al.* [11]. We evaluate Segment Lookup and CosetLkup using their released multi-threaded code. Campanelli *et al.* [11] do not release an implementation, so we implement their protocol ourselves.

5.2 Evaluation of Complete Inference Proof

We first conduct microbenchmarks of the new protocols designed for complete inference proofs, including token-to-embedding lookups, positional encoding, and decoding.

5.2.1 Performance of token-to-embedding Lookup

We evaluate our preprocessing-based matrix lookup for token-to-embedding on the GPT-2 embedding table [59] ($N = 50{,}257$, $d = 768$), varying the number of queried tokens T. We compare against general lookup scheme Logup [32] used in zkGPT [58], and matrix lookups, including CosetLkup [24], Segment Lookup [21], and Campanelli *et al.* [11].

Preprocessing costs. Table 1 reports per-table preprocessing overhead of different works. CosetLkup is not included, since it does not support preprocessing. Our scheme incurs the lowest preprocessing time (759.91 s), reducing cost by 11.5× versus Segment Lookup (8,747.70 s) and by 14.9× versus Campanelli *et al.* (11,296.96 s). It also lowers auxiliary storage costs, making preprocessing more practical.

Table 1: One-time preprocessing cost for embedding-table lookup (GPT-2, $N = 50{,}257$, $d = 768$, 32 threads).

Method	Preprocessing (s)	Size (GB)
Segment Lookup [21]	8,747.70	6.50
Campanelli et al. [11]	11,296.96	48.00
Ours	**759.91**	4.52

Online costs. Table 2 reports online costs of different works. Since LogUp [32] does not natively support *matrix* lookup, we emulate embedding retrieval by repeating its size-N lookup d times. The results show the proving time is impractical. Non-preprocessing approaches CosetLkup incur costs that scale with the table size, resulting in prohibitive prover time for this large table. In contrast, preprocessing-based methods make online costs practical by removing the table-dependent work. Table 2 shows that our method is consistently the fastest prover: for example, at $T = 2^5$ we reduce proving time to 0.035 s, compared to 4.98 s for Segment Lookup and 1.12 s for Campanelli *et al.*, yielding 142.3× and 32× speedups, respectively; at $T = 2^{10}$, our prover time remains 0.40 s. In terms of verification, Campanelli *et al.* achieve the lowest verifier runtime at about 2.5 ms. While our verifier is slightly slower in comparison, it remains efficient, with verification time consistently below 30 ms.

Our preprocessing costs across different embedding tables. We also benchmark the preprocessing cost of our matrix lookup for embedding table across representative models. Table 3 reports the preprocess time and the memory usage. For GPT-2–scale tables ($N = 50{,}257$, $d \in [768, 1600]$), preprocessing completes in less than 30 minutes and produces 4.52–9.40 GB of auxiliary data. As we scale to larger models, the cost grows accordingly: GPT-3's $d = 12288$ table requires 11,735 seconds and 72.03 GB, while DeepSeek-scale tables reach 8264 seconds / 48.05 GB (V2) and 14,505 seconds / 84.05 GB (V3). Since this preprocessing is performed once per fixed model and amortized across all subsequent inference proofs, even the performance for the largest embedding tables remain practical as a one-time cost and are modest compared to the cost of model training.

5.2.2 Performance of Positional Encoding

Absolute positional embeddings can be expressed in arithmetic circuits as a simple offset addition, incurring negligible overhead. Therefore, we focus our microbenchmark evaluation on the more complex Rotary Positional Embedding (RoPE) with $H = 12$ heads, varying the sequence length T and head dimension d_h. We compare against the naïve arithmetization proposed in zkLLM [70], which treats RoPE as matrix multiplications. Table 4 reports prover and verifier time. Our optimized approach reduces RoPE proving time by 11.5×–31.3×, while leaving verification essentially un-

Table 2: Embedding matrix lookup benchmark with GPT-2 ($N = 50257, d = 768$).

Num of Query		2^5	2^6	2^7	2^8	2^9	2^{10}
Prove Time (s)	Ours	0.035	0.045	0.063	0.10	0.19	0.40
	LogUp [32]	123.56	126.50	130.12	136.02	142.17	156.56
	CosetLkup [24]	1602.69	1604.95	1611.45	1617.64	1604.41	1612.59
	Segment Lookup [21]	4.98	5.33	6.46	9.75	14.96	25.25
	Campanelli et al. [11]	1.12	1.21	1.33	1.63	2.22	3.46
Verify Time (s)	Ours	0.022	0.023	0.023	0.024	0.027	0.030
	LogUp [32]	9.78	10.08	10.49	11.21	11.51	12.12
	CosetLkup [24]	0.0090	0.0090	0.0090	0.0090	0.0090	0.0090
	Segment Lookup [21]	0.03	0.04	0.04	0.03	0.03	0.03
	Campanelli et al. [11]	0.0024	0.0025	0.0025	0.0025	0.0025	0.0025

Table 3: Preprocessing cost of our embedding-table lookup. N is the vocabulary size and d is the embedding dimension.

Model	Embedding table		Preprocess	
	N	d	time (s)	size (GB)
GPT-2	50257	768	759.913	4.52
TinyLlama	32000	2048	929.842	6.01
GPT-2 Medium	50257	1024	984.767	6.02
GPT-2 XL	50257	1600	1536.261	9.40
DeepSeek V2	102400	4096	8264.414	48.05
GPT-3	50257	12288	11734.969	72.03
DeepSeek V3	129280	7168	14505.447	84.05

Table 4: RoPE proving and verification time ($H = 12$), comparing naïve matrix multiplications [70] against our scheme.

T	d_h	zkLLM [70]		Ours	
		Prove (s)	Verify (ms)	Prove (s)	Verify (ms)
32	64	0.31	2.28	0.027	1.90
64	64	0.62	2.26	0.046	2.11
128	64	1.22	2.27	0.079	2.15
256	64	2.44	2.41	0.16	2.25
512	64	4.90	3.37	0.31	3.47
32	128	1.22	2.27	0.044	2.02
64	128	2.43	2.40	0.078	2.03
128	128	4.87	2.64	0.16	2.25
256	128	9.70	2.78	0.31	2.27
512	128	19.49	3.51	0.63	2.82

changed, highlighting the benefit of exploiting RoPE's structure rather than arithmetizing it as matrix multiplications.

To evaluate the overhead in the real-world scenario, we also evaluate the performance of our optimization in proving Llama-2-7B [74], reducing the RoPE prover time from 95.70s to 3.06s per layer, saving 92.64s. Across 32 layers, this amounts to a total saving of $2,964.48$s.

5.2.3 Performance of Decoding

Let N denote the vocabulary size and T the context length. Decoding overhead is dominated by lookup arguments: we incur $O(TN)$ range-check lookups on logit differences for comparisons. For sampling-based decoding, the dominant extra cost is proving exponentiation/Softmax: top-k only needs Softmax over the selected set S (cost $O(T|S|)$), whereas top-p requires normalization to certify the nucleus condition (cost $O(TN)$). Other components are natively handled by the GKR circuit. We microbenchmark the lookup overhead on GPT-2 ($N = 50{,}257$) with a 2^{16} size table, comparing our *Shout* backend against the lookup in zkGPT [58]. Across the query sizes induced by TN, our scheme reduces prover time by $10.7\times$–$24.4\times$ for $T = 32$–512, primarily by lowering the constant factors in lookup proving via a more efficient backend.

5.3 One-shot transcript proof

We benchmark transcript-level proving and verification under varying final context lengths. We fix the user prompt length to $T_0 = 30$ tokens and vary the final transcript length T, so each instance contains $(T - T_0)$ model-generated tokens. We compare our one-shot transcript proof against a step-by-step approach that proves each step using prior zkLLM systems [14, 58, 70]. Our transformer proof implementation[4] builds on zkGPT [58] and incorporates Shout [63] to optimize nonlinear operations, significantly reducing prover overhead.

Table 5 shows that stepwise systems scale poorly: costs grow significantly with the number of generated tokens. For example, zkGPT's proving time rises from 65.01s at T=32 to 149,192.6s at T=512 (about 41.4 hours), and verification grows from 2.96s to 4,361.09s. Our measured zkGPT verification is slightly higher than the original report in their paper

[4] To enable an apples-to-apples comparison with prior implementations that omit non-transformer components, we report only transformer forward time. Our circuit implements the full GPT-2 block (including bias and residual connections) and, due to circuit fusion [58], also includes the final LayerNorm. The public zkGPT implementation omits bias and residuals, so our reported time is conservative.

Table 5: Transformer proving/verification for transcript generation with a fixed T_0=30 prompt and varying final length T

Transcript		zkGPT [58]		Ours	
T	$T-T_0$	Prove (s)	Verify (s)	Prove (s)	Verify (s)
32	2	65.01	2.96	26.27	0.20
64	34	1727.84	103.91	37.53	0.21
128	98	7880.97	457.47	79.68	0.27
256	226	33867.87	1358.88	180.69	0.32
512	482	149192.6	4361.09	508.46	0.51

because we include verification for lookup argument, which is omitted in their original implementation and evaluation.

ZKML [14] is prohibitively slow even for the shortest transcript (T=32), taking 7,148.4s to prove; we could not run longer contexts due to excessive prover time. At T=32, our prover is 272× faster, while achieving verifier speedups of 168×. The performance of zkML may further degrade as the context length increases. For zkLLM [70], since no GPT-2 implementation is available, we use the per-step costs reported in zkGPT [58] (Prove 15.8s, Verify 0.54s) as an estimation, scaled by $(T-T_0)$. At T=512, this gives $15.8 \times 482 \approx$ 7,615.6s proving and $0.54 \times 482 \approx 260.3$s verification, implying our one-shot approach is about $7{,}615.6/618.66 \approx 12.3\times$ faster in proving and $260.3/0.70 \approx 372\times$ faster in verification. Moreover, the zkLLM results are obtained on NVIDIA A100 GPUs, while our experiments are conducted on CPU, making the comparison conservative in favor of zkLLM [70].

In contrast, our one-shot design amortizes the entire transcript into a *single* proof. Our transformer proving time grows gently from 26.27s at T=32 to 508.46s at T=512, corresponding to $508.46/482 \approx 1.05$s per generated token, while our verification remains nearly constant (0.20–0.51s) because only one proof is verified. Our one-shot proof size grows from 22.69 MB at T=32 to 108.23 MB at T=512.[5]

5.4 End-to-end performance of zkAgent

To show the end-to-end practicality beyond microbenchmarks, we evaluate zkAgent on two realistic tool-using tasks.

Weather Agent. We run an agent that fetches weather data via a web request, consumes the returned observation, and produces a 154-token transcript. The tool authenticity proof corresponds to the zkTLS-backed binding of the web observation, while the schema proof validates the tool-call structure. The resulting end-to-end proof is 42.55 MB.

Coding Assistant. We run an agent that invokes a deterministic computation tool (Python) and incorporate the execution result into the context, producing a 146-token transcript. As above, we prove both tool authenticity and schema compliance; the end-to-end proof size is 42.38 MB.

[5]The proof size is dominated by the commitment to a size-ℓn sparse polynomial used to prove ℓ lookups into a sized-n table in Shout. With KZH as the backend, this entails a commitment size of $\sqrt{\ell n}$, which contributes significantly to the overall proof size. This could be mitigated by using Dory [42], which achieves $O(1)$ commitment size.

Table 6: End-to-end proving and verification time breakdown for two tool-using zkAgent tasks.

Component	Weather agent		Coding assistant	
	Prove (s)	Verify (s)	Prove (s)	Verify (s)
Inference	230.14	0.32	225.63	0.31
Schema	2.87	0.033	2.78	0.034
Tool	3.25	0.020	3.90	0.15
Total	236.26	0.37	232.31	0.49

Analysis. Table 6 summarizes the end-to-end breakdown across three components: the inference proof, the schema proof, and the tool-authenticity proof. For both tasks, zkAgent completes proving in under 240 s, while verification finishes in under 0.5 s and proof sizes remain around 42 MB for both traces, indicating a end-to-end proof is practical.

6 Related Works

Zero-knowledge proofs (ZKPs) for machine learning have progressed from early verifiable decision trees [86] to LLMs [58, 70]. A dominant line of work relies on SNARK protocol proving where a typically quantized model is expressed over finite fields and optimized via specialized gadgets and compilers, enabling practical proofs for common architectures and workloads (e.g., zkCNN [45], vCNN [43], verfCNN [57] for CNN inference; compiler-oriented systems such as Zen [27] and ZKML [14]; zkRNN [84] for RNN, zkLLM [70], zkGPT [58] for transformer). A primary focus is handling non-linear layers efficiently: many approaches use polynomial piecewise approximations [45, 60] or lookup arguments [14, 58, 70] trading numerical fidelity for smaller circuits. Existing approaches often restrict their scope to specific operations or building blocks of the inference pipeline. In contrast, zkAgent enables verification of full agent executions including complete inference and external tool invocations. Moreover, these works do not support efficient batched proof of multiple inferences. In particular, zkPytorch [80] briefly mentions the possibility of batched inference proofs; however, it neither accounts for the autoregressive nature of LLM nor provides a concrete construction. In contrast, zkAgent realizes a secure and efficient one-shot transcript proof.

In parallel, recent VOLE-based and other interactive proof approaches also show scalable proof techniques for non-linear functions and neural-network inference [33, 46, 77], but these protocols often impose substantially higher communication and verification overheads than SNARKs. Complementary to approximation-centric zkML, ZIP [60] demonstrates high-precision (IEEE-754) ZK inference for non-linear activations.

References

[1] Sahar Abdelnabi, Kai Greshake, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In Maura Pintor, Xinyun Chen, and Florian Tramèr, editors, *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security, AISec 2023, Copenhagen, Denmark, 30 November 2023*, pages 79–90. ACM, 2023.

[2] Anthropic. Beyond permission prompts: making claude code more secure and autonomous. Anthropic Engineering, 2025. Describes sandboxed command execution (filesystem/network isolation) in Claude Code. Published Oct 20, 2025. Accessed 2026-01-15.

[3] Anthropic. How to implement tool use | claude api docs. Documentation, 2026. Accessed: 2026-02-01.

[4] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.

[5] Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, Eran Tromer, and Madars Virza. SNARKs for C: Verifying program executions succinctly and in zero knowledge. In *Proceedings of the 33rd Annual International Cryptology Conference*, CRYPTO '13, pages 90–108, 2013.

[6] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. Aurora: Transparent succinct arguments for R1CS. In *Proceedings of the 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, EUROCRYPT '19, pages 103–128, 2019. Full version available at https://eprint.iacr.org/2018/828.

[7] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. Succinct non-interactive zero knowledge for a von Neumann architecture. In *Proceedings of the 23rd USENIX Security Symposium*, USENIX Security '14, pages 781–796, 2014. Extended version at http://eprint.iacr.org/2013/879.

[8] Brave Software and Coral Authors. Coral: Bridging parsing and zero-knowledge proofs. Blog post and open-source implementation, 2025. Accepted to IEEE S&P 2026; https://brave.com/blog/coral/, code: https://github.com/eniac/coral.

[9] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 2020.

[10] Benedikt Bünz and Binyi Chen. Protostar: Generic efficient accumulation/folding for special-sound protocols. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 77–110. Springer, 2023.

[11] Matteo Campanelli, Antonio Faonio, Dario Fiore, Tianyu Li, and Helger Lipmaa. Lookup arguments: Improvements, extensions and applications to zero-knowledge decision trees. In Qiang Tang and Vanessa Teague, editors, *Public-Key Cryptography - PKC 2024 - 27th IACR International Conference on Practice and Theory of Public-Key Cryptography, Sydney, NSW, Australia, April 15-17, 2024, Proceedings, Part II*, volume 14602 of *Lecture Notes in Computer Science*, pages 337–369. Springer, 2024.

[12] Matteo Campanelli, Dario Fiore, and Rosario Gennaro. Natively compatible super-efficient lookup arguments and how to apply them. *J. Cryptol.*, 38(1):14, 2025.

[13] Yuhan Cao, Yu Wang, Sitong Liu, Miao Li, Yixin Tao, and Tianxing He. Pay for the second-best service: A game-theoretic approach against dishonest llm providers. *arXiv preprint arXiv:2511.00847*, 2025.

[14] Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. ZKML: an optimizing system for ML inference in zero-knowledge proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys 2024, Athens, Greece, April 22-25, 2024*, pages 560–574. ACM, 2024.

[15] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. Hyperplonk: Plonk with linear-time prover and high-degree custom gates. In *Proceedings of the 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques*, EUROCRYPT '23, pages 499–530, Cham, 2023. Springer Nature Switzerland.

[16] Xinghao Chen, Zhijing Sun, Wenjin Guo, Miaoran Zhang, Yanjun Chen, Yirong Sun, Hui Su, Yijie Pan, Dietrich Klakow, Wenjie Li, and Xiaoyu Shen. Unveiling the key factors for distilling chain-of-thought reasoning. *arXiv preprint arXiv:2502.18001*, 2025. ACL 2025 Findings.

[17] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Noah Vesely, and Nicholas Ward. Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In *Proceedings of the 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, EUROCRYPT '20, 2020.

[18] Alessandro Chiesa and Eran Tromer. Proof-carrying data and hearsay arguments from signature cards. In *Proceedings of the 1st Symposium on Innovations in Computer Science*, ICS '10, pages 310–331, 2010.

[19] Alessandro Chiesa and Eran Tromer. Proof-carrying data: Secure computation on untrusted platforms (high-level description). *The Next Wave: The National Security Agency's review of emerging technologies*, 19(2):40–46, 2012.

[20] Stephen Chong, Eran Tromer, and Jeffrey A. Vaughan. Enforcing language semantics using proof-carrying data. ePrint 2013/513, 2013.

[21] Arka Rai Choudhuri, Sanjam Garg, Aarushi Goel, Sruthi Sekar, and Rohit Sinha. Sublonk: Sublinear prover plonk. *Proc. Priv. Enhancing Technol.*, 2024(3):314–335, 2024.

[22] Marco De Rossi, Davide Crapis, Jordan Ellis, and Erik Reppel. Erc-8004: Trustless agents. https://eips.ethereum.org/EIPS/eip-8004, August 2025. Draft ERC, accessed: 2026-01-14.

[23] DeepSeek-AI. Deepseek-v3 technical report. *CoRR*, abs/2412.19437, 2024.

[24] Zijing Di, Lucas Xia, Wilson D. Nguyen, and Nirvan Tyagi. Muxproofs: Succinct arguments for machine computation from vector lookups. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part V*, volume 15488 of *Lecture Notes in Computer Science*, pages 236–265. Springer, 2024.

[25] Liam Eagen, Dario Fiore, and Ariel Gabizon. cq: Cached quotients for fast lookups. *IACR Cryptol. ePrint Arch.*, 2022.

[26] Dankrad Feist and Dmitry Khovratovich. Fast amortized KZG proofs. Cryptology ePrint Archive, Paper 2023/033, 2023.

[27] Boyuan Feng, Lianke Qin, Zhenfei Zhang, Yufei Ding, and Shumo Chu. ZEN: An optimizing compiler for verifiable, zero-knowledge neural network inferences. Cryptology ePrint Archive, Paper 2021/087, 2021.

[28] Ariel Gabizon and Zachary J. Williamson. plookup: A simplified polynomial protocol for lookup tables. *IACR Cryptol. ePrint Arch.*, 2020.

[29] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. ePrint 2019/953, 2019.

[30] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. Delegating computation: Interactive proofs for Muggles. In *Proceedings of the 40th Annual ACM Symposium on Theory of Computing*, STOC '08, pages 113–122, New York, NY, USA, 2008. Association for Computing Machinery.

[31] Google. Code execution | gemini api. Google AI for Developers Documentation, 2026. Gemini code execution tool; executes code in Python. Last updated 2026-01-13 (UTC). Accessed 2026-01-15.

[32] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. *IACR Cryptol. ePrint Arch.*, 2022.

[33] Meng Hao, Hanxiao Chen, Hongwei Li, Chenkai Weng, Yuan Zhang, Haomiao Yang, and Tianwei Zhang. Scalable zero-knowledge proofs for non-linear functions in machine learning. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 3819–3836, 2024.

[34] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*, 2020.

[35] B. Hutton et al. Json schema: A media type for describing json documents. IETF Internet-Draft, 2022.

[36] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2704–2713, 2018.

[37] George Kadianakis, Arantxa Zapico, Hossein Hafezi, and Benedikt Bünz. KZH-fold: Accountable voting from sublinear accumulation. Cryptology ePrint Archive, Paper 2025/144, 2025.

[38] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *Proceedings of the 16th International Conference on the Theory and Application of Cryptology and Information Security*, ASIACRYPT '10, pages 177–194, 2010.

[39] Sehoon Kim, Amir Gholami, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. I-BERT: Integer-only BERT quantization. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pages 5506–5518, 2021.

[40] Abhiram Kothapalli and Srinath T. V. Setty. Neutronnova: Folding everything that reduces to zero-check. *IACR Cryptol. ePrint Arch.*, page 1606, 2024.

[41] Abhiram Kothapalli, Srinath T. V. Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part IV*, CRYPTO'2022, 2022.

[42] Jonathan Lee. Dory: Efficient, transparent arguments for generalised inner products and polynomial commitments. In Kobbi Nissim and Brent Waters, editors, *Theory of Cryptography - 19th International Conference, TCC 2021, Raleigh, NC, USA, November 8-11, 2021, Proceedings, Part II*, volume 13043 of *Lecture Notes in Computer Science*, pages 1–34. Springer, 2021.

[43] Seunghwa Lee, Hankyung Ko, Jihye Kim, and Hyunok Oh. vcnn: Verifiable convolutional neural network based on zk-snarks. *IEEE Trans. Dependable Secur. Comput.*, 21(4):4254–4270, 2024.

[44] Xinzhe Li. A review of prominent paradigms for llm-based agents: Tool use (including rag), planning, and feedback learning, 2024.

[45] Tianyi Liu, Xiang Xie, and Yupeng Zhang. zkcnn: Zero knowledge proofs for convolutional neural network predictions and accuracy. In Yongdae Kim, Jong Kim, Giovanni Vigna, and Elaine Shi, editors, *CCS '21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021*, pages 2968–2985. ACM, 2021.

[46] Tao Lu, Haoyu Wang, Wenjie Qu, Zonghui Wang, Jinye He, Tianyang Tao, Wenzhi Chen, and Jiaheng Zhang. An efficient and extensible zero-knowledge proof framework for neural networks. Cryptology ePrint Archive, 2024.

[47] Carsten Lund, Lance Fortnow, Howard J. Karloff, and Noam Nisan. Algebraic methods for interactive proof systems. *Journal of the ACM*, 39(4):859–868, 1992.

[48] Mitchell P. Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. Building a large annotated corpus of english: The penn treebank. *Computational Linguistics*, 19(2):313–330, 1993.

[49] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. arXiv:1609.07843, 2016.

[50] OpenAI. Data analysis with chatgpt. OpenAI Help Center, 2026. Describes ChatGPT's secure code execution environment and passing generated code for execution. Accessed 2026-01-15.

[51] OpenAI. Function calling | openai api. Documentation, 2026. Accessed: 2026-02-01.

[52] OpenAI and Stripe. Agentic commerce protocol. https://developers.openai.com/commerce/guides/get-started, January 2026. Accessed: 2026-01-14.

[53] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022.

[54] PAIML and contributors. Depyler: Compiles python to rust. GitHub repository, 2025. https://github.com/paiml/depyler.

[55] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Nghia The Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. arXiv:1606.06031, 2016.

[56] Jim Posen and Assimakis A. Kattis. Caulk+: Table-independent lookup arguments. *IACR Cryptol. ePrint Arch.*, page 957, 2022.

[57] Wenjie Qu, Yanpei Guo, Yue Ying, and Jiaheng Zhang. VerfCNN, optimal complexity zkSNARK for convolutional neural networks. Cryptology ePrint Archive, Paper 2025/2020, 2025.

[58] Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. zkgpt: An efficient non-interactive zero-knowledge proof framework for LLM inference. In Lujo Bauer and Giancarlo Pellegrino, editors, *34th USENIX Security Symposium, USENIX Security 2025, Seattle, WA, USA, August 13-15, 2025*, pages 2045–2063. USENIX Association, 2025.

[59] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[60] Arman Riasi, Haodi Wang, Rouzbeh Behnia, Viet Vo, and Thang Hoang. Zero-knowledge ai inference with

high precision. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, pages 1053–1067, 2025.

[61] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an lm-emulated sandbox. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.

[62] Srinath Setty. Spartan: Efficient and general-purpose zksnarks without trusted setup. In *Proceedings of the 40th Annual International Cryptology Conference*, CRYPTO '20, pages 704–737, 2020.

[63] Srinath Setty and Justin Thaler. Twist and shout: Faster memory checking arguments via one-hot addressing and increments. Cryptology ePrint Archive, Paper 2025/105, 2025.

[64] Srinath Setty, Justin Thaler, and Riad Wahby. Unlocking the lookup singularity with lasso. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 180–209. Springer, 2024.

[65] Peter Steinberger and OpenClaw Community. Openclaw — personal ai assistant. https://openclaw.ai/.

[66] Jianlin Su, Murtadha H. M. Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[67] Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2021.

[68] Succinct Labs. Sp1 documentation: Introduction. Documentation, 2025. https://docs.succinct.xyz/docs/sp1/introduction.

[69] Succinct Labs and contributors. Sp1: A zero-knowledge virtual machine for proving rust/llvm program execution. GitHub repository, 2025. https://github.com/succinctlabs/sp1.

[70] Haochen Sun, Jason Li, and Hongyang Zhang. zkllm: Zero knowledge proofs for large language models. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, pages 4405–4419. ACM, 2024.

[71] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.

[72] Justin Thaler. Time-optimal interactive proofs for circuit evaluation. In *Annual Cryptology Conference*, pages 71–89, Berlin, Heidelberg, 2013. Springer, Springer Berlin Heidelberg.

[73] TLSNotary Project. Tlsnotary: Secure data verification. Project website, 2025. https://tlsnotary.org/.

[74] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[75] Paul Valiant. Incrementally verifiable computation or proofs of knowledge imply time/space efficiency. In *Proceedings of the 5th Theory of Cryptography Conference*, TCC '08, pages 1–18, 2008.

[76] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 2017.

[77] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. Mystique: Efficient conversions for zero-knowledge proofs with applications to machine learning. In Michael D. Bailey and Rachel Greenstadt, editors, *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*, pages 501–518. USENIX Association, 2021.

[78] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *Conference on Language Modeling (COLM)*, 2024.

[79] Tiacheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. Libra: Succinct zero-knowledge proofs with optimal prover computation. In *Advances in Cryptology – CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part III*, page 733–764, Berlin, Heidelberg, 2019. Springer-Verlag.

[80] Tiancheng Xie, Tao Lu, Zhiyong Fang, Siqi Wang, Zhenfei Zhang, Yongzheng Jia, Dawn Song, and Jiaheng Zhang. zkpytorch: A hierarchical optimized compiler

for zero-knowledge machine learning. *IACR Cryptol. ePrint Arch.*, page 535, 2025.

[81] Xiang Xie, Kang Yang, Xiao Wang, and Yu Yu. Lightweight authentication of web data via garble-then-prove. In Davide Balzarotti and Wenyuan Xu, editors, *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024*. USENIX Association, 2024.

[82] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[83] Arantxa Zapico, Vitalik Buterin, Dmitry Khovratovich, Mary Maller, Anca Nitulescu, and Mark Simkin. Caulk: Lookup arguments in sublinear time. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*, pages 3121–3134. ACM, 2022.

[84] Fatemeh Zarinjouei, Behzad Abdolmaleki, Maryam Zarezadeh, Bhavish Mohee, Aysajan Abidin, and Stefan Köpsell. zkRNN: Zero-knowledge proofs for recurrent neural network inference. Cryptology ePrint Archive, Paper 2026/073, 2026.

[85] Fan Zhang, Ethan Cecchetti, Kyle Croman, Ari Juels, and Elaine Shi. Town crier: An authenticated data feed for smart contracts. In *Proceedings of the 2016 aCM sIGSAC conference on computer and communications security*, 2016.

[86] Jiaheng Zhang, Zhiyong Fang, Yupeng Zhang, and Dawn Song. Zero knowledge proofs for decision tree predictions and accuracy. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*, pages 2039–2053. ACM, 2020.

[87] Jiaheng Zhang, Tianyi Liu, Weijie Wang, Yinuo Zhang, Dawn Song, Xiang Xie, and Yupeng Zhang. Doubly efficient interactive proofs for general arithmetic circuits with linear prover time. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 159–177, New York, NY, USA, 2021. Association for Computing Machinery.

[88] Tianyu Zhang, Yupeng Ouyang, and Yupeng Zhang. Dynark: Making groth16 dynamic. *IACR Cryptol. ePrint Arch.*, page 1897, 2025.

A Inference Process of Decoder-only LLMs

We briefly review the inference process of decoder-only Large Language Models.

Step 1: Tokenization. Given a prompt string, a **tokenizer** deterministically converts it into a sequence of token IDs $\boldsymbol{x} = (x_0, x_1, \ldots, x_{t-1})$, where $x_i \in \mathcal{V}$ and we view the vocabulary as an index set $\mathcal{V} = [|\mathcal{V}|]$. Tokenization is model-independent: it is fully specified by the tokenizer algorithm and vocabulary, and does not involve the proprietary model weights. In most deployed LLM systems, the tokenizer is treated as public model configuration[6]. Accordingly, we treat the resulting token sequence $\boldsymbol{x} \in \mathcal{V}^t$ as part of the public transcript.

Step 2: Token-to-embedding lookup. Let $\boldsymbol{W}_E \in \mathbb{F}^{|\mathcal{V}| \times d}$ be the embedding table. The initial hidden states $\boldsymbol{H}_0 \in \mathbb{F}^{t \times d}$ are obtained by row selection:

$$\boldsymbol{H}_0 := \boldsymbol{W}_E(\boldsymbol{x}) = (\boldsymbol{W}_{E,x_0,:}, \ldots, \boldsymbol{W}_{E,x_{t-1},:}),$$

i.e., $\boldsymbol{H}_{0,i,:} = \boldsymbol{W}_{E,x_i,:}$ for all $i \in [t]$.

Step 3: Positional encoding. Decoder-only LLMs inject position information using either absolute or rotary positional encodings.

Absolute positional embeddings (APE) If the model uses APE (e.g., GPT-2 [59]), a learnable positional matrix $\boldsymbol{P} \in \mathbb{F}^{T_{\max} \times d}$ is added to embeddings:

$$\boldsymbol{H}_0 \leftarrow \boldsymbol{H}_0 + \boldsymbol{P}_{[t],:}.$$

Rotary positional embeddings (RoPE) For RoPE-based models (e.g., Llama-2 [74], Qwen [4]), no positional matrix is added at this stage. Instead, position is injected inside attention by rotating the Query and Key vectors. For example, for a query vector at position $m \in [t]$, denoted $\boldsymbol{q}_m \in \mathbb{F}^d$, the rotated vector is

$$\boldsymbol{q}_m^{\text{rot}} = \boldsymbol{R}^d_{\Theta,m} \boldsymbol{q}_m,$$

where $\boldsymbol{R}^d_{\Theta,m}$ is block-diagonal:

$$\boldsymbol{R}^d_{\Theta,m} = \begin{bmatrix} \boldsymbol{R}^{(0)}_{\Theta,m} & & \\ & \ddots & \\ & & \boldsymbol{R}^{(d/2-1)}_{\Theta,m} \end{bmatrix}$$

where each sub-matrix $R^{(i)}_{\Theta,m}$ equals

$$\begin{pmatrix} \cos m\theta_i & -\sin m\theta_i \\ \sin m\theta_i & \cos m\theta_i \end{pmatrix}$$

$$\Theta = \left\{ \theta_i = 10000^{-2i/d} \ : \ i \in [d/2] \right\}$$

[6] LLM manufacturers often release the tokenizer to facilitate token counting and billing for third-party platforms

Step 4: Stacked Transformer blocks. The core computation is performed by L Transformer blocks. Let $\boldsymbol{H}_\ell \in \mathbb{F}^{t\times d}$ denote the hidden states after ℓ blocks, with $\boldsymbol{H}_0$ as above. For each block $\ell \in [L]$, we compute

$$\boldsymbol{H}'_{\ell+1} = \boldsymbol{H}_\ell + \mathrm{MSA}_\ell(\mathrm{LN}_\ell(\boldsymbol{H}_\ell)), \tag{18}$$

$$\boldsymbol{H}_{\ell+1} = \boldsymbol{H}'_{\ell+1} + \mathrm{FFN}_\ell(\mathrm{LN}'_\ell(\boldsymbol{H}'_{\ell+1})). \tag{19}$$

Layer normalization (LN) LayerNorm is applied independently to each token position. For a row vector $\boldsymbol{h} \in \mathbb{R}^d$, define

$$\mu(\boldsymbol{h}) := \frac{1}{d}\sum_{j\in[d]} \boldsymbol{h}_j, \qquad \sigma^2(\boldsymbol{h}) := \frac{1}{d}\sum_{j\in[d]} (\boldsymbol{h}_j - \mu(\boldsymbol{h}))^2,$$

$$\mathrm{LN}(\boldsymbol{h}) := \boldsymbol{\gamma} \circ \frac{\boldsymbol{h} - \mu(\boldsymbol{h})\boldsymbol{1}}{\sqrt{\sigma^2(\boldsymbol{h}) + \varepsilon}} + \boldsymbol{\beta},$$

where $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^d$ are learned affine parameters, $\varepsilon > 0$ is a fixed constant, and $\circ$ denotes the Hadamard product. Applying LN to a matrix $\boldsymbol{H} \in \mathbb{R}^{t\times d}$ is row-wise: $(\mathrm{LN}(\boldsymbol{H}))_{i,:} = \mathrm{LN}(\boldsymbol{H}_{i,:})$ for all $i \in [t]$.

Feed-forward network (FFN) Each block also contains a position-wise MLP applied independently to each token. A common form is a two-layer MLP with an activation ϕ:

$$\mathrm{FFN}(\boldsymbol{X}) := \phi(\boldsymbol{X}\boldsymbol{W}_1 + \boldsymbol{1}\boldsymbol{b}_1^\top)\boldsymbol{W}_2 + \boldsymbol{1}\boldsymbol{b}_2^\top,$$

where $\boldsymbol{W}_1 \in \mathbb{R}^{d\times d_{\mathrm{ff}}}$, $\boldsymbol{W}_2 \in \mathbb{R}^{d_{\mathrm{ff}}\times d}$ and $\boldsymbol{b}_1 \in \mathbb{R}^{d_{\mathrm{ff}}}$, $\boldsymbol{b}_2 \in \mathbb{R}^d$. Modern LLMs often use gated variants such as SwiGLU or GeLU.

Multi-head self-attention (MSA) Let $\boldsymbol{X} = \mathrm{LN}_\ell(\boldsymbol{H}_\ell) \in \mathbb{F}^{t\times d}$. MSA forms queries/keys/values via learned projections:

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}_{Q,\ell}, \qquad \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_{K,\ell}, \qquad \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_{V,\ell}, \tag{20}$$

where $\boldsymbol{W}_{Q,\ell}, \boldsymbol{W}_{K,\ell}, \boldsymbol{W}_{V,\ell} \in \mathbb{F}^{d\times d}$. Split into h heads with head dimension $d_h = d/h$: $\boldsymbol{Q} = [\boldsymbol{Q}^{(0)}; \ldots; \boldsymbol{Q}^{(h-1)}]$ and similarly for $\boldsymbol{K}, \boldsymbol{V}$. For each head $i \in [h]$, scaled dot-product attention is

$$\boldsymbol{A}^{(i)} = \mathrm{softmax}\left(\frac{\boldsymbol{Q}^{(i)}(\boldsymbol{K}^{(i)})^\top}{\sqrt{d_h}}\right), \tag{21}$$

$$\boldsymbol{Z}^{(i)} = \boldsymbol{A}^{(i)}\boldsymbol{V}^{(i)}, \tag{22}$$

The head outputs are concatenated and projected back:

$$\mathrm{MSA}_\ell(\boldsymbol{X}) = \mathrm{Concat}(\boldsymbol{Z}^{(0)}, \ldots, \boldsymbol{Z}^{(h-1)})\boldsymbol{W}_{O,\ell}, \tag{23}$$

with $\boldsymbol{W}_{O,\ell} \in \mathbb{F}^{d\times d}$.

Step 5: Logits and next-token selection (decoding). After L blocks, the model maps hidden states to vocabulary logits via a linear head. For the last position $t-1$, let $\boldsymbol{H}_{L,t-1,:} \in \mathbb{F}^d$ be the final hidden state vector and $\boldsymbol{W}_{\mathrm{head}} \in \mathbb{F}^{d\times|\mathcal{V}|}$ be the output projection. The logits are

$$\boldsymbol{z}_{t-1} = \mathrm{LN}_{\mathrm{out}}(\boldsymbol{H}_{L,t-1,:}) \cdot \boldsymbol{W}_{\mathrm{head}}^\top \in \mathbb{F}^{|\mathcal{V}|}.$$

In deployed systems, decoding typically converts logits to a categorical distribution using a *temperature* parameter $\tau > 0$:

$$p_\tau(v) := \Pr[x_t = v \mid \boldsymbol{z}_{t-1}] = \frac{\exp(\boldsymbol{z}_{t-1}[v]/\tau)}{\sum_{u\in\mathcal{V}} \exp(\boldsymbol{z}_{t-1}[u]/\tau)}, \qquad v \in \mathcal{V}.$$

Smaller τ sharpens the distribution, while larger τ flattens it.

Greedy decoding The greedy rule deterministically selects the most likely token:

$$x_t = \arg\max_{v\in\mathcal{V}} \boldsymbol{z}_{t-1}[v] = \arg\max_{v\in\mathcal{V}} p_\tau(v),$$

which can be viewed the temperature as $\tau \to 0$.

Temperature sampling Alternatively, one may sample the next token as $x_t \sim p_\tau(\cdot)$.

Top-k sampling Let S_k be the set of k tokens with the largest probabilities under p_τ. Define the truncated distribution

$$\tilde{p}_k(v) := \begin{cases} \dfrac{p_\tau(v)}{\sum_{u\in S_k} p_\tau(u)} & v \in S_k, \\ 0 & \text{otherwise}, \end{cases}$$

and random sample $x_t \sim \tilde{p}_k(\cdot)$.

Top-p (nucleus) sampling [34] Sort tokens by decreasing probability under p_τ and let S_p be the *smallest* set whose cumulative probability mass exceeds a threshold $p \in (0,1]$: $\sum_{v\in S_p} p_\tau(v) \geq p$. Then renormalize on S_p (as top-k sampling) and sample x_t from the resulting distribution. Top-p adaptively chooses the candidate set size, unlike fixed-size top-k.

Step 6: Autoregressive loop. Generation is **autoregressive**: given the current context $\boldsymbol{x} = (x_0, \ldots, x_{t-1})$, Step 5 yields x_t, which is appended to form a longer context $(x_0, \ldots, x_t)$. The process repeats to generate $x_{t+1}, x_{t+2}, \ldots$ until a stopping condition holds (e.g., an EOS token or a maximum length). Practical implementations may maintain cached state (e.g., KV cache), but the functional semantics remain: each newly generated token is the decoding result computed from the model logits on the preceding context.

B Protocol

B.1 Sumcheck Protocol

Sumcheck Protocol is one of the most impactful interactive proofs in zero-knowledge proofs. The sumcheck problem is to sum a multilinear polynomial $f : \mathbb{F}^n \to \mathbb{F}$ on the boolean hypercube equals the claimed value v, i.e. $v = \sum_{\boldsymbol{x}\in\{0,1\}^n} f(x_0, \ldots, x_{n-1})$. Directly computing it requires time exponential in n, which is unacceptable for resource-bounded verifiers. Lund et al. [47] proposed the sumcheck protocol, which enables the verifier $\mathcal{V}$ to delegate the computation to a more powerful prover $\mathcal{P}$. The proof size of the sumcheck protocol is $O(dn)$, where d is the variable degree of f, as sends a univariate polynomial of degree d in each round. The

PROTOCOL 1 ([47]). *SumCheck Protocol*

Prover $\mathcal{P}$ claims $v = \sum_{\boldsymbol{x}\in\{0,1\}^n} f(\boldsymbol{x})$ to Verifier $\mathcal{V}$, and $\mathcal{V}$ has the oracle access to f. Let $f_0 = v$.

- In the k-th round where $1 \le k \le n-1$:
  - $\mathcal{P}$ sends a univariate polynomial $f_k(X_k) = \sum_{\boldsymbol{x}\in\{0,1\}^{n-k}} f(r_1,\ldots,r_{k-1},X_k,\boldsymbol{x})$ to $\mathcal{V}$.
  - $\mathcal{V}$ checks $f_{k-1} = f_k(0) + f_k(1)$. If the check passes, $\mathcal{V}$ sends a challenge $r_k \in \mathbb{F}$ to $\mathcal{P}$ and sets $f_k = f_k(r_k)$.
- In the n-th round:
  - $\mathcal{P}$ sends a univariate polynomial $f_n(X_n) = f(r_1,\ldots,r_{n-1},X_n)$ to $\mathcal{V}$.
  - $\mathcal{V}$ checks $f_{n-1} = f_n(0) + f_n(1)$. If the check passes, $\mathcal{V}$ sends a challenge $r_n \in \mathbb{F}$, and accepts if $f_n(r_n) = f(r_1,\ldots,r_n)$ using one oracle call to f.

prover time of the protocol is $O(dn)$ with dynamic programming proposed in [72,79]. The verifier time of the protocol is $O(dn)$ plus the time to evaluate the polynomial f at a random point. The sumcheck protocol is complete and sound with soundness error $\varepsilon = \frac{dn}{|\mathbb{F}|}$. We provide a detailed description of the sumcheck protocol in Protocol 1.

B.2 ML-friendly GKR Protocol

Let C be a depth-d arithmetic circuit arranged into layers indexed by $i \in \{0,\ldots,d\}$, where layer 0 is the output layer and layer d is the input layer. We pad each layer so that it contains 2^{s_i} gates for some $s_i \in \mathbb{N}$, and identify gates in layer i with Boolean labels $z \in \{0,1\}^{s_i}$.

Let $V_i : \{0,1\}^{s_i} \to \mathbb{F}$ map a gate label to its value at layer i, and let $\tilde{V}_i : \mathbb{F}^{s_i} \to \mathbb{F}$ denote the multilinear extension (MLE) of V_i.

Cross-layer value views $V_{i,j}$. For each $i < j$, layer i may use values originating from layer j. We index the subset of layer-j values used by layer i by $\{0,1\}^{s_{i,j}}$ (padding if needed), and define a mapping

$$\pi_{i,j} : \{0,1\}^{s_{i,j}} \to \{0,1\}^{s_j}$$

that specifies which layer-j gate value is referenced. Define

$$V_{i,j}(u) := V_j(\pi_{i,j}(u)), \quad u \in \{0,1\}^{s_{i,j}},$$

and let $\tilde{V}_{i,j}$ be the MLE of $V_{i,j}$.

Selection predicates $\widetilde{C}_{i,j}$. Define the Boolean selector

$$C_{i,j} : \{0,1\}^{s_{i,j}} \times \{0,1\}^{s_j} \to \{0,1\}, \quad C_{i,j}(u,x) = \mathbf{1}[x = \pi_{i,j}(u)],$$

and let $\tilde{C}_{i,j} : \mathbb{F}^{s_{i,j}} \times \mathbb{F}^{s_j} \to \mathbb{F}$ denote its MLE. Then for any $r \in \mathbb{F}^{s_{i,j}}$,

$$\tilde{V}_{i,j}(r) = \sum_{x\in\{0,1\}^{s_j}} \tilde{C}_{i,j}(r,x) \cdot \tilde{V}_j(x). \tag{24}$$

This is the identity used to reduce claims about $\tilde{V}_{i,j}$ to claims about $\tilde{V}_j$ via sumcheck.

Unifying dimensions via padding. For a fixed layer $i < d$, we define

$$S_i := \max_{j\in\{i+1,\ldots,d\}} s_{i,j},$$

and view all summations in the layer-i check over $\{0,1\}^{S_i}$. For each $j > i$, define the *padding indicator*

$$\widetilde{pad}_{i,j}(y) := \prod_{t=s_{i,j}}^{S_i-1} y_t, \quad y \in \{0,1\}^{S_i}.$$

When $\widetilde{pad}_{i,j}(y) = 1$, the suffix $y[s_{i,j} : S_i]$ is forced to all ones, so $y[0 : s_{i,j}]$ effectively ranges over $\{0,1\}^{s_{i,j}}$. This trick lets us sum different-$s_{i,j}$ terms within a single sumcheck instance.

Wiring predicates for layer constraints. We assume (as in [87]) that each gate in layer i has two inputs, where we designate the first input as coming from the "adjacent" view $V_{i,i+1}$, and the second input as coming from $V_{i,j}$ for some $j \in \{i+1,\ldots,d\}$.

For $i < d$ and $j \in \{i+1,\ldots,d\}$, define Boolean wiring predicates over the hypercube:

$$add_{i+1,j} : \{0,1\}^{s_i} \times \{0,1\}^{s_{i+1}} \times \{0,1\}^{s_{i,j}} \to \{0,1\},$$

$$mult_{i+1,j} : \{0,1\}^{s_i} \times \{0,1\}^{s_{i+1}} \times \{0,1\}^{s_{i,j}} \to \{0,1\},$$

where $add_{i+1,j}(z,x,u) = 1$ iff the gate z in layer i is an addition gate whose first input is the x-th entry in $V_{i,i+1}$ and whose second input is the u-th entry in $V_{i,j}$; similarly for $mult_{i+1,j}$. Let

$$\widetilde{add}_{i+1,j}, \widetilde{mult}_{i+1,j}$$

denote the multilinear extensions of $add_{i+1,j}$ and $mult_{i+1,j}$, respectively. These polynomials are fully determined by the circuit description and are efficiently evaluable by $\mathcal{V}$ at arbitrary field points.

Layer relation. For any $g \in \mathbb{F}^{s_i}$ and any $i < d$, we have

$$\begin{aligned}\tilde{V}_i(g) = &\sum_{\substack{x\in\{0,1\}^{s_{i+1}}\\ y\in\{0,1\}^{S_i}}} \sum_{j=i+1}^{d} \widetilde{pad}_{i,j}(y) \cdot \widetilde{add}_{i+1,j}\big(g,x,y[0:s_{i,j}]\big) \cdot \\ &\qquad \big(\tilde{V}_{i,i+1}(x) + \tilde{V}_{i,j}(y[0:s_{i,j}])\big) \\ &+ \sum_{\substack{x\in\{0,1\}^{s_{i+1}}\\ y\in\{0,1\}^{S_i}}} \sum_{j=i+1}^{d} \widetilde{pad}_{i,j}(y) \cdot \widetilde{mult}_{i+1,j}\big(g,x,y[0:s_{i,j}]\big) \cdot \\ &\qquad \tilde{V}_{i,i+1}(x) \cdot \tilde{V}_{i,j}(y[0:s_{i,j}]).\end{aligned} \tag{25}$$

Eq. (25) is checked via sumcheck at each layer. The formal protocol is presented in Protocol 2.

PROTOCOL 2 ([87]). *GKR Protocol for General Arithmetic Circuits*

Let C be a depth-d general arithmetic circuit as in Appendix B.2. The prover $\mathcal{P}$ wants to convince the verifier $\mathcal{V}$ that $\mathsf{out} = C(\mathsf{in})$. We assume $|\mathsf{out}|$ and $|\mathsf{in}|$ are powers of 2 (padding if necessary). Let $\tilde{V}_i$ and $\tilde{V}_{i,j}$ denote the MLEs defined in Appendix B.2, with $\tilde{V}_0$ and $\tilde{V}_d$ being the MLEs of out and in, respectively.

1. **Initialization.** $\mathcal{V}$ samples a random point $g^{(0)} \in \mathbb{F}^{s_0}$, computes or queries an oracle for $\tilde{V}_0(g^{(0)})$ from the claimed output, and sends $g^{(0)}$ to $\mathcal{P}$.
2. **Output-layer check (layer 0).** $\mathcal{P}$ and $\mathcal{V}$ run the sumcheck protocol to verify Eq. (25) for $i = 0$ at point $g^{(0)}$. At the end of sumcheck, $\mathcal{V}$ obtains a constant number of random points at which it needs evaluations of $\tilde{V}_{0,1}$ and $\tilde{V}_{0,j}$ (for $j > 1$). If the final sumcheck identity fails, $\mathcal{V}$ outputs rej.
3. **For each layer $i = 1, \ldots, d$:**
   (a) **Reducing claims targeting layer i.** By the end of the previous steps, $\mathcal{V}$ holds a collection of claims of the form $\tilde{V}_{k,i}(r^{(k,i)})$ for $k \in \{0, \ldots, i-1\}$, and possibly an additional claim $\tilde{V}_{i-1,i}(r^{(i-1,i)'})$. $\mathcal{V}$ samples random coefficients
   $$\{\alpha_{k,i}\}_{k=0}^{i-1} \cup \{\alpha'_{i-1,i}\} \subset \mathbb{F}$$
   and sends them to $\mathcal{P}$. Then $\mathcal{P}$ and $\mathcal{V}$ run sumcheck on the selector identity in Eq. (24):
   $$\sum_{k=0}^{i-1} \alpha_{k,i} \tilde{V}_{k,i}\big(r^{(k,i)}\big) + \alpha'_{i-1,i} \tilde{V}_{i-1,i}\big(r^{(i-1,i)'}\big) = \sum_{x \in \{0,1\}^{s_i}} \tilde{V}_i(x) \cdot \Big(\sum_{k=0}^{i-1} \alpha_{k,i} \tilde{C}_{k,i}\big(r^{(k,i)}, x\big) + \alpha'_{i-1,i} \tilde{C}_{i-1,i}\big(r^{(i-1,i)'}, x\big) \Big).$$
   If the sumcheck passes, $\mathcal{V}$ obtains a new random point $g^{(i)} \in \mathbb{F}^{s_i}$ and a claim $\tilde{V}_i(g^{(i)})$.
   (b) **Layer check (if $i < d$).** If $i < d$, $\mathcal{P}$ and $\mathcal{V}$ run sumcheck to verify Eq. (25) for this i at point $g^{(i)}$. At the end, $\mathcal{V}$ obtains the required evaluations of $\tilde{V}_{i,i+1}$ and $\tilde{V}_{i,j}$ (for $j > i+1$) at random points determined by the sumcheck transcript, and checks the final identity. If any check fails, output rej.
4. **Input-layer check.** At the end of the above, $\mathcal{V}$ holds a claim $\tilde{V}_d(g^{(d)})$ for some $g^{(d)} \in \mathbb{F}^{s_d}$. $\mathcal{V}$ computes $\tilde{V}_d(g^{(d)})$ locally from in or queries the input commitment/oracle and accepts iff it matches the claim.

C Preprocessing Lookup

This appendix reviews the cached-quotients (cq) *preprocessed* lookup argument of Eagen et al. [25], which we use as the table-dependent preprocessing primitive throughout the paper. The construction combines Haböck's logarithmic-derivative reduction [32] with univariate KZG polynomial commitments [38] and a one-time cache that makes the *table-side* quotient check efficient in the online phase.

Indexing and domains. For an integer n, write $[n] := \{0, 1, \ldots, n-1\}$. Let $\mathbb{F}$ be a prime field with $\mathrm{char}(\mathbb{F}) > \max(N, \ell)$, and let $\mathbb{F}^* := \mathbb{F} \setminus \{0\}$ denote the multiplicative group of nonzero field elements. For $d \in \mathbb{N}$, let $\mathbb{F}_{<d}[X]$ denote the set of univariate polynomials over $\mathbb{F}$ of degree strictly less than d. Assume N, ℓ are powers of two (pad otherwise). Let $V, H \subset \mathbb{F}^*$ be multiplicative subgroups with $|V| = N$ and $|H| = \ell$, generated by ω_V and ω_H:

$$V = \{\omega_V^j\}_{j \in [N]}, \quad H = \{\omega_H^i\}_{i \in [\ell]}.$$

Their vanishing polynomials are

$$Z_V(X) = \prod_{x \in V} (X - x) = X^N - 1, \quad Z_H(X) = \prod_{x \in H} (X - x) = X^\ell - 1.$$

Statement. Given a table vector $\boldsymbol{b} = (b_j)_{j \in [N]} \in \mathbb{F}^N$ and a query vector $\boldsymbol{a} = (a_i)_{i \in [\ell]} \in \mathbb{F}^\ell$, we want to prove multiset containment

$$\{a_i\}_{i \in [\ell]} \subseteq \{b_j\}_{j \in [N]}.$$

Equivalently, there exists a multiplicity vector $\boldsymbol{m} = (m_j)_{j \in [N]} \in \mathbb{F}^N$, where $\boldsymbol{m}$ is the number of times b_j occurs in $\boldsymbol{a}$ such that the following logarithmic-derivative identity holds as rational functions:

$$\sum_{i \in [\ell]} \frac{1}{a_i + X} = \sum_{j \in [N]} \frac{m_j}{b_j + X}. \tag{26}$$

Intuitively, both sides represent the same multiset of linear factors $(X + a_i)$, with the RHS grouping equal values via multiplicities.

Polynomial encoding and commitments. Let $T(X) \in \mathbb{F}_{<N}[X]$ be the unique polynomial interpolating the table on V, and $F(X) \in \mathbb{F}_{<\ell}[X]$ the unique polynomial interpolating the witness on H:

$$T(\omega_V^j) = b_j \ \ (j \in [N]), \quad F(\omega_H^i) = a_i \ \ (i \in [\ell]).$$

We use univariate KZG commitments and write $\mathsf{Commit}(\cdot)$ for commitments, together with the standard opening interface. We rely on the usual $\mathbb{F}$-linearity of KZG commitments.

A subgroup-sum identity. We use the univariate sumcheck from Aurora [6] that for a multiplicative subgroup $S \subset \mathbb{F}^*$ of size t and any polynomial $p \in \mathbb{F}_{<t}[X]$,

$$\sum_{x \in S} p(x) \;=\; t \cdot p(0). \tag{27}$$

This follows from $\sum_{x \in S} x^k = 0$ for $1 \le k \le t-1$ and $\sum_{x \in S} 1 = t$.

Log-derivative reduction at a random point. The verifier samples $\beta \xleftarrow{\$} \mathbb{F}$. Except with negligible probability over β, it suffices to check Eq. (26) at $X = \beta$. Define pointwise reciprocals on H and V:

$$A(\omega_H^i) := \frac{1}{F(\omega_H^i) + \beta} = \frac{1}{a_i + \beta},$$
$$B(\omega_V^j) := \frac{m_j}{T(\omega_V^j) + \beta} = \frac{m_j}{b_j + \beta}.$$

Let $A(X) \in \mathbb{F}_{<\ell}[X]$ interpolate $\{A(\omega_H^i)\}_{i \in [\ell]}$ on H, and $B(X) \in \mathbb{F}_{<N}[X]$ interpolate $\{B(\omega_V^j)\}_{j \in [N]}$ on V. Then Eq. (26) at $X = \beta$ is equivalent to the grand-sum equality

$$\sum_{x \in H} A(x) \;=\; \sum_{x \in V} B(x), \tag{28}$$

which by Eq. (27) reduces to a single scalar constraint:

$$N \cdot B(0) \;=\; \ell \cdot A(0). \tag{29}$$

Well-formedness via divisibility. It remains to enforce that A and B are consistent with T, F and a multiplicity polynomial. Let $M(X) \in \mathbb{F}_{<N}[X]$ interpolate $\boldsymbol{m}$ on V, i.e., $M(\omega_V^j) = m_j$. The definitions of A and B imply that the following polynomials vanish on V and H, respectively, and hence are divisible by the corresponding vanishing polynomials:

$$A(X) \cdot (F(X) + \beta) - 1 = Q_A(X) \cdot Z_H(X). \tag{30}$$
$$B(X) \cdot (T(X) + \beta) - M(X) = Q_B(X) \cdot Z_V(X), \tag{31}$$

Given KZG commitments, the verifier can check these divisibility relations succinctly using pairing.

The preprocessing interface. We now specify the offline data produced by $\mathsf{Preproc}(\boldsymbol{b})$ and used throughout the main text. Given a table $\boldsymbol{b} \in \mathbb{F}^N$ and its interpolant $T \in \mathbb{F}_{<N}[X]$ over V), the preprocessing algorithm outputs

$$\mathsf{Preproc}(\boldsymbol{b}) \;=\; \mathsf{pp}(\boldsymbol{b}),$$

which consists of the following public data for a fixed KZG SRS and domain (V, Z_V):

- Lagrange-basis commitments on V. Commitments to the Lagrange basis polynomials $(L_j(X))_{j \in [N]}$, where $L_j(\omega_V^r) = \mathbf{1}[r = j]$. These allow the prover to commit to any polynomial supported on a sparse set $I \subseteq [N]$ using $O(|I|)$ group operations via linear combination.
- Auxiliary commitments for openings at 0. Commitments to $\Delta L_j(X) := \frac{L_j(X) - L_j(0)}{X}$ for all $j \in [N]$, which are convenient for efficient openings at $X = 0$ when the polynomial is sparse.
- Table commitment. The commitment to the table polynomial $c_T := \mathsf{Commit}(T)$.
- Cached-quotient commitments. Cached-quotient commitments $(c_{Q,j})_{j \in [N]}$, where each $Q_j(X)$ is defined by

$$L_j(X) \cdot T(X) \;=\; b_j \cdot L_j(X) \;+\; Z_V(X) \cdot Q_j(X). \tag{32}$$

  Such Q_j exists because $L_j(x)T(x) = b_j L_j(x)$ for all $x \in V$, hence $L_j(X)T(X) - b_j L_j(X)$ vanishes on V and is divisible by $Z_V(X)$.

Among the above outputs, the Lagrange-basis and ΔL_j commitments depend only on (N, V) and can be reused across many tables of size N, while c_T and $(c_{Q,j})_{j \in [N]}$ are table-dependent.

Linearity. For fixed SRS, $\mathsf{pp}(\cdot)$ is $\mathbb{F}$-linear in the table entries, i.e., $\mathsf{pp}(\alpha \boldsymbol{b} + \alpha' \boldsymbol{b}') = \alpha\, \mathsf{pp}(\boldsymbol{b}) + \alpha'\, \mathsf{pp}(\boldsymbol{b}')$. This is the property used in the main text to fold multiple preprocessed tables via random linear combinations.

In our applications, the multiplicity vector $\boldsymbol{m}$ is sparse: only indices actually appearing in the witness have $m_j \neq 0$. Let $I := \mathsf{supp}(\boldsymbol{m}) \subseteq [N]$ with $|I| \le \ell$. Writing A in the Lagrange basis on V,

$$A(X) = \sum_{j \in I} A(\omega_V^j)\, L_j(X),$$

one can check by linearity that the corresponding quotient in Eq. (31) satisfies

$$Q_A(X) = \sum_{j \in I} A(\omega_V^j)\, Q_j(X). \tag{33}$$

Therefore, the prover can form the quotient *commitment* as

$$\mathsf{Commit}(Q_A) = \sum_{j \in I} A(\omega_V^j)\, \mathsf{Commit}(Q_j),$$

using only $O(|I|) = O(\ell)$ group operations, without interpolating or dividing any degree-N polynomials online.

Complexity. Let ℓ be the number of queries and N the table size.

- **One-time preprocessing (per fixed table):** $\mathsf{Preproc}(\boldsymbol{b})$ costs $O(N \log N)$ field/group operations, leveraging fast quotient/commitment techniques [26, 88]. The stored table-dependent data is $O(N)$ group elements.
- **Online proving:** table-side work is dominated by $O(|\mathsf{supp}(\boldsymbol{m})|) = O(\ell)$ group operations to form commitments to sparse M, A and the quotient commitment $\mathsf{Commit}(Q_A)$. Query-side work is $O(\ell \log \ell)$ field operations to form Q_B on H, plus $O(\ell)$ commitment/opening work.
- **Verification:** a constant number of pairing checks, plus $O(1)$ field operations for Eq. (29).

Crucially, the *online* complexity is essentially independent of the large table size N.

D Prefix Equivalence of Teacher-Forced Causal Forward Pass

This appendix formalizes the correctness of the one-shot proving strategy in Section 4.1. We show that, for a decoder-only Transformer with causal masking, a single teacher-forced forward pass on the full transcript produces intermediate-position logits identical to those obtained by an online autoregressive execution on the corresponding prefix. This equivalence justifies using teacher-forced logits to enforce the shifted next-token constraints in Eq. (17).

Setup. Let $\mathsf{tr} = (t_0, \ldots, t_{T-1}) \in [N]^T$ be a token sequence over a vocabulary of size N. For any $n \in [T]$, consider running the decoder-only Transformer in inference mode on the length-n prefix $\mathsf{tr}[0:n] := (t_0, \ldots, t_{n-1})$. This produces a logit matrix $\boldsymbol{L}^{(n)} \in \mathbb{F}^{n \times N}$, whose i-th row is the logit vector at position i, denoted $\boldsymbol{\ell}_i^{(n)} \in \mathbb{F}^N$, i.e., $\boldsymbol{L}^{(n)}[i,:] = \boldsymbol{\ell}_i^{(n)}$.

We rely on the following standard architectural properties of decoder-only Transformers. They are stated explicitly to make the prefix-induction argument self-contained.

1. **Prefix-consistent inputs.** The embedding lookup and all position-dependent terms (absolute/rotary encodings or biases) for positions $0, \ldots, n-1$ depend only on their indices, hence are identical whether we run length n or any longer length $n' > n$.
2. **Positionwise non-attention maps.** Every non-attention sublayer (e.g., LayerNorm, MLP, output projections) is applied independently to each position, with residual additions performed entrywise.
3. **Causal masking.** Self-attention is causally masked: position i assigns zero weight to keys $j > i$.

We state the mask in the form implemented in the circuit (see Eq. (16) in Section 4.1). Consider a single attention head. Given $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{F}^{n \times d_h}$, define scores $s_{i,j} := \langle \boldsymbol{q}_i, \boldsymbol{k}_j \rangle / \sqrt{d_h}$ and weights

$$\alpha_{i,j} := \begin{cases} \dfrac{\exp(s_{i,j})}{\sum_{k \le i} \exp(s_{i,k})}, & j \le i, \\ 0, & j > i, \end{cases} \tag{34}$$

and output rows

$$\big(\mathsf{Attn}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V})\big)[i,:] := \sum_{j=0}^{n-1} \alpha_{i,j} \boldsymbol{V}[j,:]. \tag{35}$$

Multi-head attention concatenates heads and applies a positionwise linear map, so it suffices to argue headwise.

Lemma 1 (Prefix invariance of causally masked attention)**.** *Let $m > n$. Let $\boldsymbol{X}^{(m)} \in \mathbb{F}^{m \times d}$ be any input matrix, and let $\boldsymbol{X}^{(n)} := \boldsymbol{X}^{(m)}_{[n],:} \in \mathbb{F}^{n \times d}$ be its length-n prefix. Let $\mathsf{Attn}^{(r)}(\cdot)$ denote the masked attention computed on a length-r input using Eqs. (34) and (35). Then for every $i \in [n]$,*

$$\big(\mathsf{Attn}^{(m)}(\boldsymbol{X}^{(m)})\big)[i,:] = \big(\mathsf{Attn}^{(n)}(\boldsymbol{X}^{(n)})\big)[i,:].$$

Proof. Fix any $i \in [n]$. For each $r \in \{n, m\}$, write the per-position projections as

$$\boldsymbol{Q}^{(r)} = \boldsymbol{X}^{(r)} \boldsymbol{W}_Q, \quad \boldsymbol{K}^{(r)} = \boldsymbol{X}^{(r)} \boldsymbol{W}_K, \quad \boldsymbol{V}^{(r)} = \boldsymbol{X}^{(r)} \boldsymbol{W}_V,$$

and denote rows by $\boldsymbol{q}_i^{(r)}, \boldsymbol{k}_j^{(r)}, \boldsymbol{v}_j^{(r)}$. Because $\boldsymbol{X}^{(n)}$ is the prefix of $\boldsymbol{X}^{(m)}$, we have $\boldsymbol{q}_i^{(m)} = \boldsymbol{q}_i^{(n)}$ and $\boldsymbol{k}_j^{(m)} = \boldsymbol{k}_j^{(n)}$, $\boldsymbol{v}_j^{(m)} = \boldsymbol{v}_j^{(n)}$ for all $j \le i$.

Under the causal mask Eq. (34), for both lengths $r \in \{n, m\}$, the attention weights at position i satisfy $\alpha_{i,j}^{(r)} = 0$ for all $j > i$, and the normalization is over the identical set $\{0, \ldots, i\}$. Since all scores $s_{i,j}^{(r)}$ for $j \le i$ coincide across the two runs, the normalized weights $\{\alpha_{i,j}^{(m)}\}_{j \le i}$ and $\{\alpha_{i,j}^{(n)}\}_{j \le i}$ match entrywise. Plugging into Eq. (35) gives the claimed equality of output rows. □

Theorem 1 (Prefix equivalence of intermediate logits)**.** *For every $i \in [T-1]$, the teacher-forced length-T run and the online prefix run of length $i+1$ produce the same logit vector at position i:*

$$\boldsymbol{\ell}_i^{(T)} = \boldsymbol{\ell}_i^{(i+1)}.$$

Equivalently, the first $i+1$ rows of $\boldsymbol{L}^{(T)}$ equal $\boldsymbol{L}^{(i+1)}$.

Proof. We prove by induction over Transformer layers $\ell = 0, 1, \ldots, L$ that the hidden-state prefixes match. Let $\boldsymbol{H}^{(\ell,n)} \in \mathbb{F}^{n \times d}$ denote the layer-ℓ hidden states in the length-n run.

Base layer. By prefix-consistency of embedding and positional encoding, $\boldsymbol{H}^{(0,T)}_{[i+1],:} = \boldsymbol{H}^{(0,i+1)}$.

Inductive step. Assume $\boldsymbol{H}^{(\ell-1,T)}_{[i+1],:} = \boldsymbol{H}^{(\ell-1,i+1)}$. Every positionwise map (LayerNorm, MLP) preserves row-prefix equality. For the attention sublayer, apply Lemma 1 to conclude that the first $i+1$ rows of the attention output in the length-T run equal those of the length-$(i+1)$ run. Residual additions are entrywise and thus also preserve prefix equality. Therefore $\boldsymbol{H}^{(\ell,T)}_{[i+1],:} = \boldsymbol{H}^{(\ell,i+1)}$.

Logits. The final projection to logits is positionwise linear, so it preserves the same row-prefix identity. In particular, the i-th row matches: $\boldsymbol{\ell}_i^{(T)} = \boldsymbol{\ell}_i^{(i+1)}$. □

Remarks. The proof is exact under the restricted-softmax causal mask in Eq. (34). If an implementation uses a large negative finite constant as a mask, the equality becomes approximate.

Precision	WikiText-2	PTB	LAMBADA
FP32	51.20	72.68	220.29
Our quantization	51.67	74.70	226.65

Table 7: Perplexity (PPL) of FP32 vs. our quantized inference.

E Quantization and Accuracy Evaluation

Zero-knowledge proof systems operate over finite fields, whereas modern ML inference is dominated by floating-point arithmetic and non-arithmetic primitives (e.g., comparisons or exponentials). Following prior zkML systems [14, 27, 45, 58, 70], we adopt fixed-point quantization so that the entire inference can be expressed using integer arithmetic and then embedded into the field.

We represent each floating-point tensor using signed integers together with a *public* scaling factor. Concretely, a real value x is encoded as an integer $q \in \mathbb{Z}$ such that $x \approx q \cdot s$, where we choose a scale of the form $s = c \cdot 2^e$. This choice keeps rescaling lightweight and is standard in integer-only inference and quantized Transformers [36, 39].

To ensure soundness, we conservatively choose integer bitwidths so that intermediate values do not overflow under the quantized semantics. In practice, we obtain practical bounds by profiling FP32 inference and then applying a safety margin. For example, in GPT-2 inference, embeddings and residual-stream activations concentrate near zero and typically lie within a bounded range. These bounds directly determine the range-check sizes required by the proof.

LLM level. We report perplexity (PPL) in Table 7 on three standard language modeling benchmarks: WikiText-2 [49], Penn Treebank (PTB) [48], and LAMBADA [55]. Perplexity is the exponentiated average negative log-likelihood, and therefore directly measures how much the next-token distribution shifts under quantization. A small PPL increase indicates that quantization introduces only minor distortion to the model's predictive distribution.

Agent-level. Beyond standalone language modeling, we evaluate end-to-end agent executions, where small numeric perturbations may flip tool selection, termination decisions, or final answers. All runs use greedy decoding. To isolate the effect of quantization, stability metrics replay the same FP32 prompts and observations so that the agent receives identical external inputs. We report two stability metrics in Table 8: *action agreement*, the fraction of steps whose agent actions match FP32, and *first divergence*, the first step at which the action differs (set to 7 if no divergence occurs within the step budget).

Task	Action agreement (%)	First divergence (step)
HotpotQA	96.33	5.68

Table 8: Agent-level stability of quantized inference relative to FP32 on HotpotQA.

PROTOCOL 3 ([25, 26, 32]). *Preprocessed Lookup (cached-quotients)*

Let $T \in \mathbb{F}_{<N}[X]$ interpolate the table $\boldsymbol{b}$ over V and $F \in \mathbb{F}_{<\ell}[X]$ interpolate the witness $\boldsymbol{a}$ over H.

$\mathsf{Preproc}(\boldsymbol{b})$(offline). Fix a KZG SRS and domain parameters (V, Z_V).

1. Output commitments to the Lagrange basis $(\mathsf{Commit}(L_j))_{j\in[N]}$ and $(\mathsf{Commit}(\Delta L_j))_{j\in[N]}$ for sparse commitments on V.
2. Interpolate $T \in \mathbb{F}_{<N}[X]$ from $\boldsymbol{b}$ over V, and output the table commitment $c_T := \mathsf{Commit}(T)$.
3. For each $j \in [N]$, define Q_j by the cached-quotient identity Eq. (32) and output $c_{Q,j} := \mathsf{Commit}(Q_j)$.

Return $\mathsf{pp}(\boldsymbol{b}) := \big((\mathsf{Commit}(L_j)), (\mathsf{Commit}(\Delta L_j)), c_T, (c_{Q,j})_{j\in[N]}\big)$.

$\mathsf{Prove}(\mathsf{pp}(\boldsymbol{b}), \boldsymbol{a})$(online).

1. Compute a multiplicity vector $\boldsymbol{m} = (m_j)_{j\in[N]}$. Let $I := \mathsf{supp}(\boldsymbol{m}) = \{j \in [N] : m_j \neq 0\}$, which satisfies $|I| \leq \ell$. Let $M \in \mathbb{F}_{<N}[X]$ interpolate $\boldsymbol{m}$ on V. Using the precomputed Lagrange-basis commitments, compute the sparse commitment
$$c_M := \mathsf{Commit}(M) = \sum_{j\in I} m_j \,\mathsf{Commit}(L_j),$$
2. Derive a challenge β through Fiat–Shamir. Define $A \in \mathbb{F}_{<N}[X]$ and $B \in \mathbb{F}_{<\ell}[X]$ by
$$A(\omega_V^j) = \frac{m_j}{T(\omega_V^j) + \beta}, \qquad B(\omega_H^i) = \frac{1}{F(\omega_H^i) + \beta}.$$
Since $A(\omega_V^j) = 0$ for all $j \notin I$, using the precomputed commitments $(\mathsf{Commit}(L_j))_{j\in[N]}$, the prover computes
$$c_A := \mathsf{Commit}(A) = \sum_{j\in I} A(\omega_V^j)\,\mathsf{Commit}(L_j) = \sum_{j\in I} \frac{m_j}{T(\omega_V^j) + \beta}\,\mathsf{Commit}(L_j),$$
which takes $O(|I|) = O(\ell)$ group operations. $c_B := \mathsf{Commit}(B)$ is computed using the standard procedure.
3. Provide quotient commitments witnessing Eq. (31)–(30):
   - Table side: form $c_{Q_A} := \mathsf{Commit}(Q_A)$ via the cached-quotient combination Eq. (33) using $(c_{Q,j})$.
   - Witness side: compute Q_B on H (e.g., via FFT) and send $c_{Q_B} := \mathsf{Commit}(Q_B)$.
4. Open $A(0)$ and $B(0)$ with KZG proofs. For $A(0)$, exploit sparsity using the precomputed $(\mathsf{Commit}(\Delta L_j))$: the KZG opening proof at 0 can be formed as the sparse linear combination
$$\mathsf{Open}_0(A) \;=\; \mathsf{Commit}\left(\frac{A(X) - A(0)}{X}\right) = \sum_{j\in I} A(\omega_V^j)\,\mathsf{Commit}(\Delta L_j),$$
which costs $O(|I|)$ group operations. The opening of $B(0)$ is computed using the standard KZG opening procedure.

Return $\pi \;=\; \big(c_M, c_A, c_B, c_{Q_A}, c_{Q_B},\, \mathsf{Open}_0(A), \mathsf{Open}_0(B)\big)$

$\mathsf{Verify}(\mathsf{pp}(\boldsymbol{b}), c_F, \pi)$**.**

1. Recompute β via Fiat–Shamir.
2. Verify the pairing for Eq. (31) and Eq. (30).
3. Verify the openings of $A(0)$ and $B(0)$, and check the grand-sum constraint Eq. (29): $N \cdot A(0) = \ell \cdot B(0)$.

Accept iff all checks pass.